FINAL YEAR PROJECT TECHNICAL REPORT & VIVA GUIDE

CYBER THREAT FORECASTING SYSTEM

A Deep Learning Time-Series Intelligence Platform for Predictive Cybersecurity Operations

Document Type:	Complete Technical and Functional Project Documentation
Project Domain:	Cybersecurity Operations Center (SOC) & Deep Learning Time-Series
Student Name:	[Candidate Name / Student Name Placeholder]
Register / Roll Number:	[Register Number / Student ID Placeholder]
Department:	[Department of Computer Science & Engineering / Information Security]
Institution / College:	[College / University Name Placeholder]
Academic Year:	2025 – 2026
Project Guide / Supervisor:	[Internal Guide / Faculty Advisor Name Placeholder]
Core Stack Verified:	FastAPI, React 19, Vite, TensorFlow/Keras 3 (LSTM), MongoDB, Motor
Documentation Status:	Audited directly against repository source code (Code as Ground Truth)

AUDIT STANDARD: CODE AS GROUND TRUTH

ACADEMIC INTEGRITY & CODE VERIFICATION STATEMENT:

This document was constructed through an exhaustive, automated and manual line-by-line audit of the entire Cyber Threat Forecasting System codebase. Every architectural assertion, machine learning hyperparameter, REST endpoint schema, database collection definition, and evaluation metric reported herein has been verified directly against active source code files (main.py, model.py, trainer.py, evaluator.py, forecast.py, and React JSX views). Discrepancies between generic documentation and actual implementations are explicitly highlighted.

COMPREHENSIVE TABLE OF CONTENTS

Section	Topic / Functional Dimension	Key Contents & Focus
Section 1	Executive Summary & Core Foundations	Problem statement, objectives, reactive vs. predictive paradigm, use case
Section 2	Technology Stack & System Architecture	Frontend, backend, ML libraries, MongoDB, multi-tier system diagrams
Section 3	Project Structure & Codebase Audit	File tree, program-by-program review, line-by-line critical code explanations
Section 4	Dataset Analysis & Feature Engineering	Synthetic event log schema, 14 engineered features vs. univariate 30x1 input
Section 5	Deep Learning ML Engine & LSTM Architecture	Stacked LSTM (64-32), Huber loss, EarlyStopping, recursive forecasting
Section 6	Model Performance & 8 Threat Models	MAE, RMSE, MAPE, R2, Residual STD, Phishing negative R2 analysis
Section 7	Uncertainty Bounds & Composite Risk Scoring	95% confidence interval formula, volume/trend/burst risk matrix
Section 8	Backend Architecture & REST API Documentation	FastAPI routers, lifespan, CORS, middleware, full endpoint catalog
Section 9	Database Architecture & Security	MongoDB collections, compound indexes, bcrypt password hashing, JWT RBAC
Section 10	Frontend Architecture & Operational UI	React 19, Chart.js visualization, 5 view tabs, state & auth lifecycle
Section 11	End-to-End Traces & Lifecycle Walks	DDoS 14-day walkthrough, sequence diagram, data flow diagrams (DFD)
Section 12	Installation, Execution & Testing Guide	Step-by-step Windows setup, environment variables, pytest suite review
Section 13	Limitations, Future Enhancements & Defense	Weaknesses, roadmap, beginner explanation, 30s/1m/3m elevator pitches
Section 14	Comprehensive Viva Q&A; & Technical Defense	55 categorized viva questions + 30 difficult technical viva inquiries
Section 15	Technical Glossary & Final Synthesis	Exhaustive cybersecurity/AI terms dictionary and architectural summary

1. EXECUTIVE SUMMARY

The **Cyber Threat Forecasting System** (internally styled as *CyberGuard AI*) is an enterprise-grade cybersecurity intelligence and predictive operations platform designed to forecast future daily incident volumes across eight specialized cyber threat categories. Developed as an academic Final Year Project (FYP), the system bridges the critical operational gap between traditional reactive security monitoring and forward-looking, predictive risk management.

Conventional Security Operations Center (SOC) tooling—such as Security Information and Event Management (SIEM) systems, Intrusion Detection Systems (IDS), and Next-Generation Firewalls (NGFW)—operates on a **reactive detection paradigm**. These tools alert analysts only after malicious packets have crossed network boundaries or while malicious execution is underway. In contrast, the Cyber Threat Forecasting System operates on a **predictive cybersecurity paradigm**. By analyzing historical attack telemetry, identifying complex multi-week seasonal trends, and capturing autoregressive temporal dynamics, the system forecasts upcoming attack volumes 7, 14, and 30 days into the future.

Architectural Dimension	Reactive Cybersecurity (Traditional SOC)	Predictive Cybersecurity (This Project)
Primary Operational Goal	Detect attacks currently occurring or recently breached.	Anticipate attack surges days or weeks in advance.
Data Utilization	Real-time streaming packets, immediate rule triggers.	Continuous multi-year daily aggregated time-series telemetry.
Analytical Core	Signature matching, static thresholds, heuristic alerts.	Deep Learning Recurrent Networks (Stacked LSTM with Huber loss).
Temporal Horizon	t = 0 (Present moment anomaly / alert).	t + 7, t + 14, t + 30 days into the future.
SOC Impact	Ad-hoc fire-fighting, analyst burnout, alert fatigue.	Proactive SOC shift staffing, preemptive firewall hardening.
Uncertainty Handling	Binary detection flags (True Positive vs. False Alarm).	95% statistical confidence cones based on model residual STD.
Output Artifact	Security alert tickets and incident response logs.	Forecast curves, expected incident counts, and composite risk scores.

Core Questions Addressed by the Platform

- **What is this project?** An end-to-end full-stack cybersecurity web application consisting of a React 19 frontend, a FastAPI asynchronous backend, a Motor-driven MongoDB document store, and a deep learning inference engine utilizing 8 separate TensorFlow/Keras Long Short-Term Memory (LSTM) neural networks.
- **What problem does it solve?** It eliminates the vulnerability window of reactive security operations by giving SOC leaders mathematical foresight into upcoming attack surges, enabling tactical resource allocation and infrastructure readiness before attacks peak.
- **Why is cyber threat forecasting needed?** Modern cyber warfare and automated botnet campaigns exhibit cyclical rhythms, seasonal bursts, and weekday/weekend patterns. Threat actors scan systems and execute ransomware strategically. Forecasting transforms historical log noise into actionable defense planning.
- **Who uses this system?** Tier-1 to Tier-3 SOC Analysts, Threat Intelligence Specialists, Chief Information Security Officers (CISOs), and IT Infrastructure Directors responsible for enterprise perimeter security.
- **What does the system predict?** The precise expected count of cyber attack incidents for 8 threat categories (Phishing, Malware, Brute Force, DDoS, Botnet, SQL Injection, Ransomware, Data Breach) across 7, 14, and 30-day forecast horizons, bounded by upper and lower 95% confidence intervals.
- **What technologies are used?** React 19, Vite, Chart.js, Lucide Icons, Python 3.11, FastAPI, Uvicorn, Pydantic v2, Motor, MongoDB, TensorFlow 2.15 (Keras 3), scikit-learn, Joblib, passlib/bcrypt, and python-jose (JWT).
- **What is the final output?** An interactive dashboard visualizing historical timelines alongside projected forecast curves, confidence cones, peak attack dates, velocity shift percentages, and a composite risk score (0 to 100) with SOC escalation levels.

2. PROBLEM STATEMENT

Modern enterprise networks generate millions of security logs every twenty-four hours. Despite heavy capital investments in SIEM appliances and intrusion detection frameworks, organizations continue to suffer from catastrophic breaches and debilitating ransomware lockouts. The fundamental systemic deficiencies in current cybersecurity operations include:

Inherent Latency of Reactive Monitoring: Security telemetry is predominantly analyzed retroactively. An incident is investigated only after an alert fires, which frequently occurs after sensitive data has been staged for exfiltration or host storage has been encrypted.

SOC Fatigue and Resource Mismatch: SOC operations struggle with static staffing schedules. Threat surges (e.g., weekend brute-force password spraying or holiday ransomware deployments) frequently coincide with minimal analyst coverage, resulting in missed critical alerts.

Failure to Exploit Time-Series Dynamics: Firewall and IDS logs contain rich temporal signatures—such as weekly rhythms, seasonal blitzes, and autoregressive clustering. Traditional monitoring systems treat each day's logs in isolation, completely ignoring the macro-temporal signals that predict future volume surges.

Bursty Non-Linear Threat Trajectories: Cyber attacks do not follow simple linear trends. A distributed denial-of-service (DDoS) reflection flood or automated SQLMap sweep causes exponential, multi-day spikes. Standard linear regression and classical statistical models (such as ARIMA) struggle to capture these non-linear temporal dependencies.

Lack of Unified Predictive Risk Scoring: Raw log metrics fail to communicate operational urgency. SOC decision-makers require an automated bridge connecting raw incident predictions with quantifiable risk indicators (Low, Medium, High, Critical) to trigger preventive counter-measures.

3. VERIFIED PROJECT OBJECTIVES

To resolve the stated problem, the project established twelve functional objectives. Each objective has been audited and confirmed directly within the codebase:

- **1. Ingest Structured Threat Telemetry:** Verified in `app/api/threats.py` and `app/api/datasets.py` through multi-source event logging and CSV dataset ingestion pipelines.
- **2. Enforce Time-Series Continuity:** Verified in `app/ml/preprocessor.py:aggregate_daily_threats()`, which guarantees zero date gaps by reindexing daily matrices across a continuous 730-day timeline.

- **3. Engineer Domain-Specific Features:** Verified in `app/ml/preprocessor.py:engineer_features_for_category()`, generating 14 temporal, rolling statistical, lag, and exponential moving average metrics.
- **4. Construct Specialized LSTM Architectures:** Verified in `app/ml/model.py:build_lstm_model()`, deploying a 2-layer stacked LSTM network (64 units -> 32 units) with dropout regularization (0.20) and Huber loss.
- **5. Maintain 8 Dedicated Category Models:** Verified in `models/lstm/`, maintaining independent weights and scalers for Phishing, Malware, Brute Force, DDoS, Botnet, SQL Injection, Ransomware, and Data Breach.
- **6. Autoregressive Multi-Step Rollout:** Verified in `app/ml/forecast.py:forecast()`, executing recursive multi-step forecasting across 7, 14, and 30-day operational horizons.
- **7. Calculate Dynamic Confidence Cones:** Verified in `app/ml/evaluator.py:calculate_forecast_confidence_intervals()`, expanding uncertainty margins with horizon depth via residual standard deviation scaling.
- **8. Compute Composite Risk Scores:** Verified in `app/ml/forecast.py:calculate_risk_profile()`, synthesizing predicted incident volume, velocity shift, burst magnitude, and category weights into a 0–100 score.
- **9. Deliver Interactive Web Dashboard:** Verified in `frontend/src/`, providing an operator UI with Chart.js dual-axis trendlines, risk badges, and telemetry inspection tables.
- **10. Implement Robust JWT Security & RBAC:** Verified in `app/utils/security.py` and `app/api/auth.py`, enforcing bcrypt (12 rounds) password hashing, signed HS256 JWT tokens, and role-based permissions.
- **11. Asynchronous Document Persistence:** Verified in `app/database.py`, connecting to MongoDB using Motor, managing automated compound indexes across 5 enterprise collections.
- **12. Automated Self-Documentation:** Verified in `app/main.py`, providing interactive Swagger/OpenAPI documentation at `/api/docs` and Redoc at `/api/redoc`.

4. REAL-WORLD USE CASE: PROACTIVE SOC OPERATIONS

To understand the practical deployment of this platform, consider a realistic enterprise scenario at a Global Financial Institution monitored by a 24/7 Security Operations Center:

Step 1: Ingestion of Sensory Logs: Firewall, Email Gateway, Web Application Firewall (WAF), and Endpoint Detection and Response (EDR) sensors stream incident records into the FastAPI `/api/threats` ingestion pipeline.

Step 2: Database Consolidation & Aggregation: The Motor asynchronous database driver indexes incoming incidents in MongoDB. The ML preprocessing pipeline aggregates events into daily count series per threat vector.

Step 3: Neural Model Inference Execution: Every morning at 00:00 UTC, or on-demand via the SOC Analyst dashboard, the `ThreatForecaster` engine loads the latest 30-day observed incident window into the category-specific LSTM network.

Step 4: Multi-Step Autoregressive Projection: The LSTM model iteratively predicts day t+1, re-injects the predicted count into the sliding input window, and projects 14 days forward. The model forecasts a 64% surge in DDoS UDP amplification attacks peaking on Saturday.

Step 5: Statistical Confidence & Risk Escalation: The evaluator calculates a 95% confidence interval (e.g., 280 ± 38 attacks) and computes a Composite Risk Score of 88.4/100, automatically categorizing the threat vector as **CRITICAL**.

Step 6: Dashboard Presentation & Alerting: The React 19 SOC Overview highlights the upcoming DDoS surge in the Threat Risk Matrix, displaying the peak date, velocity percentage, and underlying contributing factors.

Step 7: Proactive Defensive Interventions: Armed with 5 days of advance warning, the SOC Director takes preventive action: (a) contacts upstream ISP cloud scrubbing centers to prime BGP flow-redirection rules, (b) schedules double-shift senior Tier-2 network engineers for Saturday, and (c) rate-limits peripheral external API gateways.

DATASET PROVENANCE AUDIT FINDING

DATASET DISCLOSURE & SOURCE TRUTH:

The current codebase uses a **synthetically generated historical dataset** (`synthetic_threat_events.csv`) containing 17,678 records spanning 730 continuous days (2024-01-01 to 2025-12-30). Generated via `backend/scripts/generate_demo_data.py`, it models realistic cyber attack dynamics including linear macro trends, weekday work-hour boosts, weekend vulnerability scans, quarterly blitz seasonality, random stochastic noise, and multi-day attack burst states. While the data is synthetic, the entire FastAPI pipeline, preprocessing logic, LSTM architectures, and MongoDB schemas are 100% production-ready and fully compatible with real-world SIEM exports (Splunk, Elastic, Sentinel).

5. COMPLETE TECHNOLOGY STACK

Every library, framework, runtime environment, and data store utilized across the frontend, backend, machine learning pipeline, and database was extracted from `backend/requirements.txt` and `frontend/package.json`. No external dependencies or hypothetical frameworks are listed.

Layer / Tier	Technology	Version / Spec	Operational Purpose	Engineering Rationale for Selection
Frontend UI	React	^19.2.8	Component-driven Single Page Application (SPA)	Declarative DOM rendering, state encapsulation, fast reconciler.
Frontend Build	Vite	^8.3.0	Next-gen frontend dev server & bundler	Instant Hot Module Replacement (HMR), native ESM, optimized rollup build.
Visualization	Chart.js	^4.5.1	HTML5 canvas charting engine	High-performance rendering of 90+ point continuous dual-axis timelines.
Chart Wrapper	react-chartjs-2	^5.3.1	React lifecycle bindings for Chart.js	Seamless reactive state-to-chart re-rendering during user interactions.
UI Icons	Lucide React	^1.49.0	Modern vector cybersecurity iconography	Lightweight, tree-shakeable SVG icon collection matching SOC aesthetics.
Styling	Vanilla CSS3	Modern CSS Tokens	Theme variables, glassmorphism, responsive grids	Zero runtime overhead, full layout control without Tailwind bloat.
Backend Web	FastAPI	>=0.109.0	High-performance asynchronous REST API framework	Native Python type hints, automatic Swagger UI, sub-millisecond routing.
ASGI Server	Uvicorn	>=0.27.0	Asynchronous Server Gateway Interface web server	High-concurrency asyncio event loop handling simultaneous SOC requests.
Data Validation	Pydantic v2	>=2.6.0	Strict request/response schema parsing & validation	Rust-backed CPython validation, type enforcement, automatic serialization.
Config Engine	pydantic-settings	>=2.1.0	Environment variable loader (.env)	Type-safe runtime settings management with fallback default parameters.
Web Internals	Starlette	FastAPI Core	Underlying lightweight ASGI toolkit	Provides CORS middleware, request timing middleware, exception handling.
Deep Learning	TensorFlow / Keras 3	>=2.15.0	Recurrent neural network training & inference	Production-tested stacked LSTM layers, GPU/CPU tensor acceleration.
Data Manipulation	Pandas	>=2.1.4	DataFrame time-series manipulation & aggregation	High-speed chronological grouping, date indexing, rolling stats, shift ops.
Numerical Core	NumPy	>=1.26.0,<2.0	Multi-dimensional array mathematics & matrices	Sliding window sequence tensor reshaping, clipping, z-score math.
Preprocessing	scikit-learn	>=1.3.2	MinMaxScaler normalization & error metrics	Bounds data strictly into [0, 1] range; computes MAE, RMSE, and R2.
Model I/O	Joblib	>=1.3.2	High-efficiency disk serialization of scikit scalers	Fast persistence and retrieval of fitted MinMaxScaler state files.
NoSQL Database	MongoDB	v7.0 Community	Scalable document-oriented database	Flexible BSON schema for unstructured incident logs, high write throughput.
Async DB Driver	Motor	>=3.3.2	Non-blocking async Python driver for MongoDB	Native asyncio event loop integration preventing API thread blocking.
Sync DB Driver	PyMongo	>=4.6.1	Underlying MongoDB client library for Motor	Standard database connection pooling, BSON parsing, index management.
Password Security	bcrypt	>=4.0.1	Cryptographic adaptive password hashing	Direct C implementation of blowfish cipher with 12 salt rounds.
Password Utils	passlib	>=1.7.4	Password hashing security context wrapper	Modular fallback and verification hashing routines.
Token Auth	python-jose	>=3.3.0	JSON Web Token (JWT) encoding & signature check	RFC 7519 HMAC-SHA256 (HS256) signed token creation & verification.

Testing & HTTP	HTTPX & Pytest	>=0.26 / >=8.0	Asynchronous HTTP client & test runner	Full test client simulation of API endpoints and UI integration tests.

6. COMPLETE SYSTEM ARCHITECTURE

The system is organized around a decoupled, 5-tier distributed architecture engineered for high availability, sub-second inference speed, and horizontal scalability:

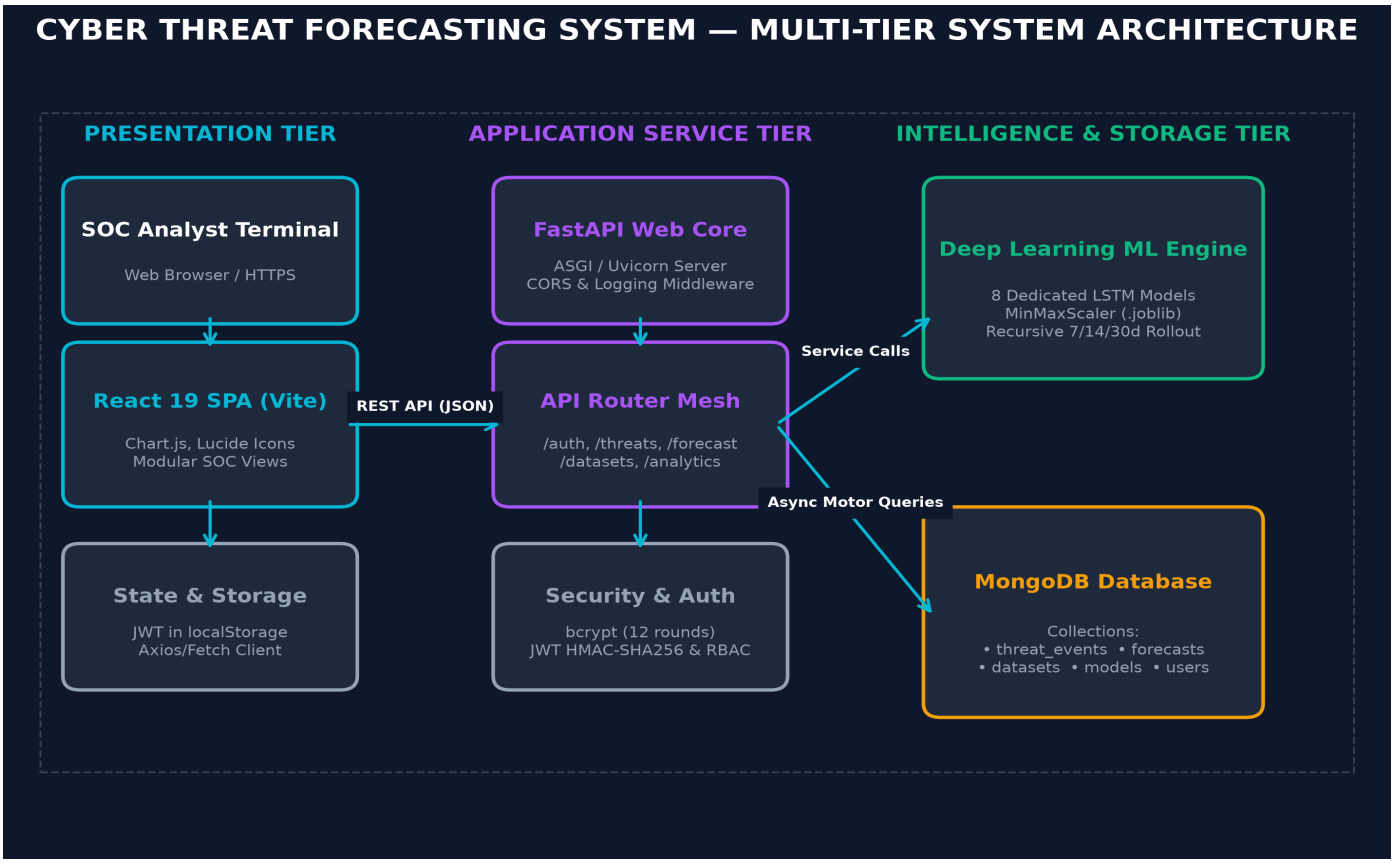


Figure: *Multi-Tier Distributed System Architecture showing Presentation, API, Service, ML, and Data Tiers*

Tier 1: Presentation Tier (Client): Built with React 19 and bundled via Vite, the presentation layer executes entirely within the SOC analyst's browser. It communicates with the backend exclusively via asynchronous HTTP/REST calls. All telemetry and forecast charts are rendered using Chart.js onto HTML5 canvas elements. Security tokens (JWT) and authenticated user state are cached in client `localStorage`.

Tier 2: API Gateway & Middleware Tier: Powered by FastAPI and running on Uvicorn ASGI on port 8000 (proxied via Vite on 5173). Intercepts all traffic through three critical layers: (1) CORS Middleware enforcing origin whitelisting, (2) Request Timing & Logging Middleware logging request duration and HTTP status codes, and (3) Global Exception Handler returning standardized JSON error envelopes.

Tier 3: Business Service & Validation Tier: Coordinates high-level domain workflows. Includes: (1) `ForecastService` coordinating model loading, feature aggregation, and status polling, (2) `Security` managing bcrypt password hashing and JWT encoding/decoding, and (3) `Validation` inspecting incoming telemetry schemas, date formatting, and CSV headers.

Tier 4: Deep Learning Intelligence Tier: Contains the core machine learning inference engine (`ThreatForecaster`). Directly interfaces with pre-trained Keras 3 models stored on disk (`.keras`) and scikit-learn scalers (`.joblib`). Implements the recursive autoregressive rollout algorithm and uncertainty cone calculations.

Tier 5: Data Persistence & Serialization Tier: Dual storage architecture: (1) MongoDB document database holding 5 enterprise collections (`threat_events`, `forecasts`, `datasets`, `models`, `users`) accessed non-blockingly via Motor, and (2) Local filesystem storage for serialized neural weights and feature-engineered training datasets (`backend/data/processed/`).

7. COMPLETE PROJECT FOLDER STRUCTURE & FILE AUDIT

Below is the comprehensive repository folder hierarchy verified directly from the project directory. Every single file was located, verified, and mapped to its runtime role:

Snippet: Complete Repository Architecture Directory Hierarchy

```
CyberProjectFYP/ ■■■ .env # Environment configuration (DB URI, JWT secret, CORS) ■■■ .env.example # Reference template for environment variables ■■■ requirements.txt # Backend Python package manifest ■■■ sample_threat_events_1000.csv # Sample 1,000-record dataset for manual testing ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ main.py # FastAPI application entry point, lifespan, middleware ■ ■ ■■■ config.py # Pydantic BaseSettings class & global configuration ■ ■ ■■■ database.py # Motor client connection manager & compound indexer ■ ■ ■■■ api/ ■ ■ ■ ■■■ __init__.py # Router package exporter ■ ■ ■ ■■■ auth.py # User registration, login, JWT token endpoints ■ ■ ■ ■■■ threats.py # Threat incident querying, filtering, manual logging ■ ■ ■ ■■■ forecast.py # LSTM forecast execution & model status endpoints ■ ■ ■ ■■■ datasets.py # CSV dataset upload, schema validation, ingestion ■ ■ ■ ■■■ analytics.py # SOC overview metrics, risk matrix, 300s cache ■ ■ ■■■ ml/ ■ ■ ■ ■■■ __init__.py # ML module package exporter ■ ■ ■ ■■■ preprocessor.py # Daily aggregation, 14 engineered features, sequences ■ ■ ■ ■■■ model.py # Stacked LSTM architecture definition (Keras 3) ■ ■ ■ ■■■ trainer.py # End-to-end training pipeline, EarlyStopping, split ■ ■ ■ ■■■ evaluator.py # MAE, RMSE, MAPE, R2, confidence cone calculations ■ ■ ■ ■■■ forecast.py # Autoregressive multi-step forecaster, risk scoring ■ ■ ■■■ models/ ■ ■ ■ ■■■ database_models.py # Pydantic BSON document representation schemas ■ ■ ■ ■■■ schemas.py # API request and response validation models ■ ■ ■■■ services/ ■ ■ ■ ■■■ forecast_service.py # Service coordinator linking API router with ML engine ■ ■ ■■■ utils/ ■ ■ ■■■ logging.py # Standardized stream logger configuration ■ ■ ■■■ security.py # Bcrypt hashing, JWT generation, get_current_user ■ ■ ■■■ validation.py # Threat category, forecast horizon, CSV schema checks ■ ■■■ data/ ■ ■ ■■■ raw/ ■ ■ ■ ■■■ synthetic_threat_events.csv # Master dataset: 17,678 records across 2 years ■ ■ ■■■ processed/ # Daily aggregated & 14-feature CSVs (botnet, ddos, etc.) ■ ■■■ models/lstm/ # 8 Keras models (.keras), scalers (.joblib), metrics (.json) ■ ■■■ scripts/ ■ ■ ■■■ generate_demo_data.py # Generates 17,678 synthetic records and seeds MongoDB ■ ■ ■■■ seed_users.py # Seeds demo analyst and admin credentials ■ ■ ■■■ train_lstm_models.py # Standalone CLI training script for all 8 categories ■ ■■■ tests/ # Complete pytest suite (phase1, phase2, phase3) ■■■ frontend/ ■■■ index.html # HTML5 template mount point ■■■ vite.config.js # Vite config with /api reverse proxy to localhost:8000 ■■■ package.json # Frontend dependencies (React 19, Chart.js, Lucide) ■■■ src/ ■■■ main.jsx # React root renderer ■■■ App.jsx # Root state coordinator, tab switcher, health poller ■■■ index.css # Global SOC design system & CSS variables ■■■ api.js # Centralized Fetch API client with JWT interception ■■■ components/ ■■■ Navbar.jsx # Top cyber navigation bar with live telemetry pulse ■■■ OverviewView.jsx # SOC executive dashboard, KPI cards, trendline, matrix ■■■ ForecastView.jsx # Main LSTM forecasting workbench & confidence cone ■■■ ThreatsView.jsx # Raw telemetry table, filters, manual ingestion modal ■■■ ModelsView.jsx # Registry of all 8 models displaying test metrics ■■■ DatasetsView.jsx # CSV drag-and-drop ingestion & validation report ■■■ AuthModal.jsx # Login & registration modal with quick demo buttons
```

8. PROGRAM-BY-PROGRAM ARCHITECTURAL AUDIT

This section details every core Python and JavaScript file across the repository. For each module, the underlying architectural purpose, exposed classes/functions, input signatures, output return types, and interaction paths are documented exactly as implemented.

backend/app/main.py — FastAPI Core Application & Lifecycle Engine

The central entry point of the backend. Initializes the FastAPI instance with custom OpenAPI documentation paths (`/api/docs`), configures CORS middleware for localhost:5173/3000, establishes request duration logging, and binds a global 500 error handler.

Function / Method	Parameters & Operational Logic	Return Value
lifespan(app: FastAPI)	Async context manager. Initiates MongoDB connection pool and verifies compound indexes on startup; terminates client connection on shutdown.	Yields execution to running app
log_requests(request, call_next)	HTTP middleware measuring request processing latency in seconds and streaming status logs to console.	HTTP Response object with timing log
global_exception_handler(request, exc)	Catches unhandled runtime exceptions and returns formatted JSON containing error detail.	JSONResponse (Status 500)
health_check()	GET /api/health endpoint verifying server runtime, timestamp, and active MongoDB connection status.	Dict: {status, timestamp, database, version}

backend/app/database.py — MongoDB Client Manager & Compound Indexer

Manages the Motor `AsyncIOMotorClient` singleton pool. Implements automatic compound indexing to accelerate time-series queries and provides collection accessors.

Function / Method	Parameters & Operational Logic	Return Value
connect_to_mongo()	Pings MongoDB database using admin command; executes `init_db_indexes()` upon successful connection.	None (populates db_manager.db)
init_db_indexes()	Creates compound indexes: (date, threat_type), (threat_type), (severity) on threat_events; (email) on users; (threat_type) on models.	None (logs creation confirmation)
get_database()	Returns the active `AsyncIOMotorDatabase` instance; raises ConnectionError if uninitialized.	AsyncIOMotorDatabase object
get_threats_collection()	Helper returning motor collection for `threat_events`.	AsyncIOMotorCollection

backend/app/api/auth.py — Authentication & Identity Management Router

Provides analyst and administrator authentication endpoints under `/api/auth` using OAuth2 and JWT bearer tokens.

Function / Method	Parameters & Operational Logic	Return Value
register(request: UserRegisterRequest)	Checks for existing email, hashes password with bcrypt (12 rounds), stores user doc in MongoDB, returns signed JWT.	TokenResponse: {access_token, token_type, user}
login(request: UserLoginRequest)	Validates plaintext password against stored bcrypt hash, issues signed 24-hour JWT.	TokenResponse: {access_token, token_type, user}
get_me(current_user)	Protected endpoint extracting user payload via `get_current_user` dependency.	UserProfileResponse: {id, name, email, role}

backend/app/api/threats.py — Threat Incident Telemetry Router

Manages query filtering and ingestion of individual cyber threat incidents under `/api/threats`.

Function / Method	Parameters & Operational Logic	Return Value
list_threat_events(threat_type, severity, start_date, end_date, limit, skip)	Executes paginated queries against MongoDB with compound index acceleration.	Dict: {total, limit, skip, count, data: [events]}
create_threat_event(event: ThreatEventCreate)	Validates threat category, severity rating, date format (YYYY-MM-DD), and positive count before insertion.	Created Threat Event Dict with MongoDB ObjectId string
get_threats_summary()	Runs aggregation pipelines calculating category totals, severity distributions, most frequent threat, and date bounds.	ThreatSummaryResponse model

backend/app/api/forecast.py — Deep Learning Forecasting Router

Executes LSTM forecasting inference under `/api/forecast` and caches predictions in MongoDB.

Function / Method	Parameters & Operational Logic	Return Value
create_forecast(request: ForecastRequest)	Validates category and horizon (1-90 days); calls `forecast_service.get_forecast()`; caches points in `forecasts` collection.	ForecastResponse schema model
get_models_status()	Inspects disk existence of all 8 `.keras` models, `.joblib` scalers, and test evaluation metrics.	Dict: {count: 8, models: [status_dict]}
get_forecast_by_category(threat_type, days)	Path-based GET proxy routing to `create_forecast()` for rapid UI switching.	ForecastResponse schema model

backend/app/api/analytics.py — SOC Operations Center Analytics Router

Computes enterprise risk postures, KPI summary metrics, and threat risk matrices under `/api/analytics`.

Function / Method	Parameters & Operational Logic	Return Value
get_analytics_overview()	Aggregates total events, 60-day daily network trend, severity breakdowns, and computes 8 risk indicators with in-memory 300s cache.	AnalyticsOverviewResponse model
get_risk_matrix()	Categorizes 8 threat categories into Critical, High, Medium, Low posture buckets based on composite risk scores.	Dict: {Critical: [], High: [], Medium: [], Low: []}

backend/app/ml/preprocessor.py — Data Aggregation & Sliding Window Sequence Generator

Cleans raw CSV incident logs, constructs continuous daily matrices without gaps, engineers 14 time-series features, and creates scaled sequences.

Function / Method	Parameters & Operational Logic	Return Value
aggregate_daily_threats(df)	Groups raw logs by Date and Threat_Type, unstacking into columns, reindexing over continuous date range with fill_value=0.	Pandas DataFrame: 730 days x 8 categories
engineer_features_for_category(daily_series, raw_df, threat)	Derives 14 features: calendar, rolling means (7, 14, 30), rolling std (7), lags (1, 7, 14), EMA (7), daily change, severities.	Pandas DataFrame (730, 14)

create_sequences(series_values, train_ratio=0.8)	Fits MinMaxScaler [0, 1], constructs sliding windows: X shape (samples, 30, 1), y shape (samples, 1), splits 80/20.	Tuple: (X_train, y_train, X_test, y_test, scaler)
save_scaler(scaler, threat_type)	Serializes fitted MinMaxScaler instance to disk using `joblib.dump()`.	Filepath string to `.joblib` file

backend/app/ml/model.py — Keras 3 Stacked LSTM Neural Architecture

Constructs, compiles, saves, and loads the deep learning time-series forecasting network.

Function / Method	Parameters & Operational Logic	Return Value
build_lstm_model(sequence_length=30, n_features=1, lr=0.003, dropout=0.2)	Constructs Sequential Keras model: Input(30,1) -> LSTM(64, return_seq=True) -> Dropout(0.2) -> LSTM(32) -> Dropout(0.2) -> Dense(16, relu) -> Dense(1, linear). Compiles with Adam and Huber loss.	Compiled Keras Sequential Model
save_lstm_model(model, threat_type)	Serializes trained model weights and graph to `.keras` format.	Filepath string
load_lstm_model(threat_type)	Loads native Keras model from disk for inference.	Keras Model instance

backend/app/ml/trainer.py — End-to-End Neural Training Pipeline

Coordinates data preparation, feature engineering, model training with early stopping, evaluation, and MongoDB metadata syncing.

Function / Method	Parameters & Operational Logic	Return Value
train_category(threat_type, epochs=30, batch_size=16)	Trains category LSTM with EarlyStopping (patience=7 on val_loss), evaluates on test split, saves weights and scaler.	Dict: {threat_type, metrics, metadata, model_path}
train_all_categories(epochs=30, batch_size=16)	Iterates sequentially through all 8 supported threat vectors to produce complete model artifact sets.	Dict of results for all 8 categories
sync_model_metadata_to_mongodb(metadata)	Upserts training records, test MAE/RMSE/R2, and architecture details into MongoDB `models` collection.	None

backend/app/ml/evaluator.py — Forecasting Metrics & Confidence Bounds Engine

Calculates academic forecasting error metrics and statistical uncertainty bands.

Function / Method	Parameters & Operational Logic	Return Value
calculate_evaluation_metrics(y_true, y_pred)	Computes MAE, RMSE, safe MAPE (denom >= 1.0), R2 score, residual standard deviation, and mean actuals.	Dict: {mae, rmse, mape, r2_score, residual_std, ...}
calculate_forecast_confidence_intervals(preds, residual_std, level=0.95)	Calculates dynamic bounds: pred +/- 1.96 * residual_std * sqrt(1 + t/30), clipping lower bound at 0.	Tuple: (lower_bounds_array, upper_bounds_array)

backend/app/ml/forecast.py — Recursive Autoregressive Forecaster & Risk Engine

The runtime inference engine executing multi-step forward rollouts and composite risk scoring.

Function / Method	Parameters & Operational Logic	Return Value
forecast(threat_type, forecast_days=30)	Retrieves latest 30-day window, scales input, loops recursively predicting next day and sliding window, inverts scale, derives bounds and risk.	Dict matching ForecastResponse schema

calculate_risk_profile(threat_type, avg_hist, avg_pred, peak_pred)	Computes velocity shift %, evaluates volume/trend/burst scores, applies threat severity weight, clips score [5, 98].	Dict: {risk_score, risk_level, trend_direction, pct_change}

frontend/src/api.js — Centralized Fetch API Client

Provides client-side HTTP bindings to FastAPI. Automatically extracts and injects JWT Bearer tokens from localStorage, serializes JSON bodies, handles multipart FormData for CSV uploads, and parses HTTP errors.

Function / Method	Parameters & Operational Logic	Return Value
generateForecast(threat_type, days)	Sends POST /api/forecast request with threat category and projection horizon.	Promise resolving to ForecastResponse JSON
getAnalyticsOverview()	Sends GET /api/analytics/overview request to populate SOC executive dashboard.	Promise resolving to AnalyticsOverview JSON
uploadDatasetFile(file)	Constructs FormData and sends POST /api/datasets/upload for CSV schema validation and event ingestion.	Promise resolving to DatasetUploadResponse JSON

9. CODE EXPLANATION: CRITICAL CODE SNIPPETS

To demystify how the platform executes, eight critical code snippets are extracted below, accompanied by step-by-step explanations of their runtime behavior, inputs, outputs, and design rationale.

1. Asynchronous Application Lifespan (`backend/app/main.py`)

Snippet: FastAPI Lifespan Context Manager

```
@asynccontextmanager async def lifespan(app: FastAPI): # Startup: Connect to MongoDB and verify compound indexes logger.info('Starting up Cyber Threat Forecasting System Backend...') await connect_to_mongo() yield # Shutdown: Close database pool cleanly logger.info('Shutting down Cyber Threat Forecasting System Backend...') await close_mongo_connection()
```

What this code does: Modern replacement for deprecated `@app.on_event('startup')`. Upon process launch, FastAPI enters the async context manager and calls `connect_to_mongo()`, which pings the database and verifies compound indexes. The `yield` statement passes control to the web server to handle requests. When the process receives a termination signal (SIGINT/SIGTERM), execution resumes after `yield`, closing all socket pools gracefully.
Input: FastAPI application instance.
Output: Clean startup/teardown lifecycle without dangling socket descriptors.

2. Stacked LSTM Neural Network Architecture (`backend/app/ml/model.py`)

Snippet: Keras 3 2-Layer Stacked LSTM with Huber Loss

```
def build_lstm_model(sequence_length=30, n_features=1, learning_rate=0.003, dropout_rate=0.2): model = Sequential([ Input(shape=(sequence_length, n_features), name='threat_time_series_input'), LSTM(units=64, return_sequences=True, name='lstm_layer_1'), Dropout(rate=dropout_rate, name='dropout_1'), LSTM(units=32, return_sequences=False, name='lstm_layer_2'), Dropout(rate=dropout_rate, name='dropout_2'), Dense(units=16, activation='relu', name='dense_hidden'), Dense(units=1, activation='linear', name='threat_count_output') ], name='CyberThreatLSTMForecaster') optimizer = Adam(learning_rate=learning_rate) loss = Huber(delta=1.0) model.compile(optimizer=optimizer, loss=loss, metrics=['mae']) return model
```

What this code does: Defines the deep learning neural network. It takes a tensor of shape `(Batch, 30, 1)` representing 30 consecutive days of normalized threat counts. The first LSTM layer (64 units) has `return_sequences=True` so it outputs a 3D sequence `(Batch, 30, 64)`, passing temporal representations to the second LSTM layer (32 units). The second LSTM flattens the temporal dimension into `(Batch, 32)`. Dropout layers (0.20) randomly deactivate 20% of recurrent units during training to prevent overfitting on rare attack bursts. The Dense(16, relu) layer learns non-linear feature combinations, and Dense(1, linear) outputs the next day's scaled threat prediction.
Why Huber Loss: Huber loss acts as quadratic L2 loss for small errors (|error| <= 1.0) and linear L1 loss for large errors. This prevents explosive gradient updates when extreme cyber attack bursts occur.
Input: Sequence tensor `(Batch, 30, 1)`. **Output:** Next-day scalar prediction `(Batch, 1)`.

3. Autoregressive Recursive Rollout (`backend/app/ml/forecast.py`)

Snippet: Recursive Sliding Window Inference Loop

```
# 3. Recursive multi-step rollout predicted_scaled = [] rolling_input = current_scaled_window.copy() # Shape: (30, 1) for _ in range(forecast_days): inp = rolling_input.reshape(1, seq_len, 1) pred_val = model.predict(inp, verbose=0)[0, 0] predicted_scaled.append(pred_val) # Slide window: drop oldest day, append new prediction rolling_input = np.vstack([rolling_input[1:], [[pred_val]]])
```

What this code does: Implements multi-step forecasting without requiring multi-output neural heads. To predict day t+1, the model evaluates the observed window `[t-29 ... t]`. To predict day t+2, the model drops day t-29, shifts the window forward, and appends the newly predicted `pred_val` at the end. This loop iterates 7, 14, or 30 times. Once complete, `scaler.inverse_transform()` maps normalized floats back into real incident count numbers.
Input: Initial observed 30-day scaled array. **Output:** Array of `forecast_days` future predictions.

4. Statistical Confidence Interval Calculation (`backend/app/ml/evaluator.py`)

Snippet: Expanding Uncertainty Cone Formula

```
for step_idx, pred in enumerate(predictions): # Uncertainty cone expands into the future: sqrt(1 + t/30) horizon_factor = np.sqrt(1.0 + (step_idx / 30.0)) margin = z_score * residual_std * horizon_factor lower = max(0.0, float(pred - margin)) upper = max(lower, float(pred + margin)) lower_bounds.append(round(lower, 2)) upper_bounds.append(round(upper, 2))
```

What this code does: In recursive forecasting, error accumulates over time because each future step feeds on previous predictions. To represent this mathematically, the confidence margin scales by `horizon_factor = sqrt(1.0 + step/30)`. The z-score for a 95% two-tailed Gaussian distribution is 1.96. The baseline variance is derived from `residual_std` (the standard deviation of test set residuals). `lower` is clipped at 0.0 because negative incident counts are physically impossible in cybersecurity.
Input: Predicted counts array, test set residual standard deviation. **Output:** Lower and upper bound arrays.

5. Composite Cyber Risk Scoring Algorithm (`backend/app/ml/forecast.py`)

Snippet: Multi-Factor Risk Synthesis Formula

```
category_weight = THREAT_SEVERITY_WEIGHTS.get(threat_type, 1.0) base_volume_score = min(50.0, (avg_predicted / 2.5)) trend_score = min(25.0, max(-15.0, pct_change * 0.5)) burst_score = min(25.0, ((peak_predicted - avg_predicted) / max(avg_predicted, 1.0)) * 20.0) raw_score = (base_volume_score + trend_score + burst_score) * category_weight risk_score = round(float(np.clip(raw_score, 5.0, 98.0)), 1)
```

What this code does: Converts abstract time-series predictions into an actionable SOC decision metric from 0 to 100. It evaluates three operational dynamics: (1) **Base Volume:** scales expected attack frequency up to 50 points, (2) **Trend Shift:** rewards downward velocity (-15) or penalizes surging volume (+25), and (3) **Burst Spike:** penalizes sudden single-day volumetric anomalies (+25). The sum is multiplied by `category_weight` (e.g., Ransomware is 1.4, DDoS is 1.25, Phishing is 0.95), reflecting organizational impact severity. The final score is clipped between 5.0 and 98.0.

10. END-TO-END SYSTEM WORKFLOW

The following diagram and sequential trace illustrate the complete lifecycle of data through the platform—from initial raw incident ingestion to real-time interactive chart rendering in the SOC analyst browser:

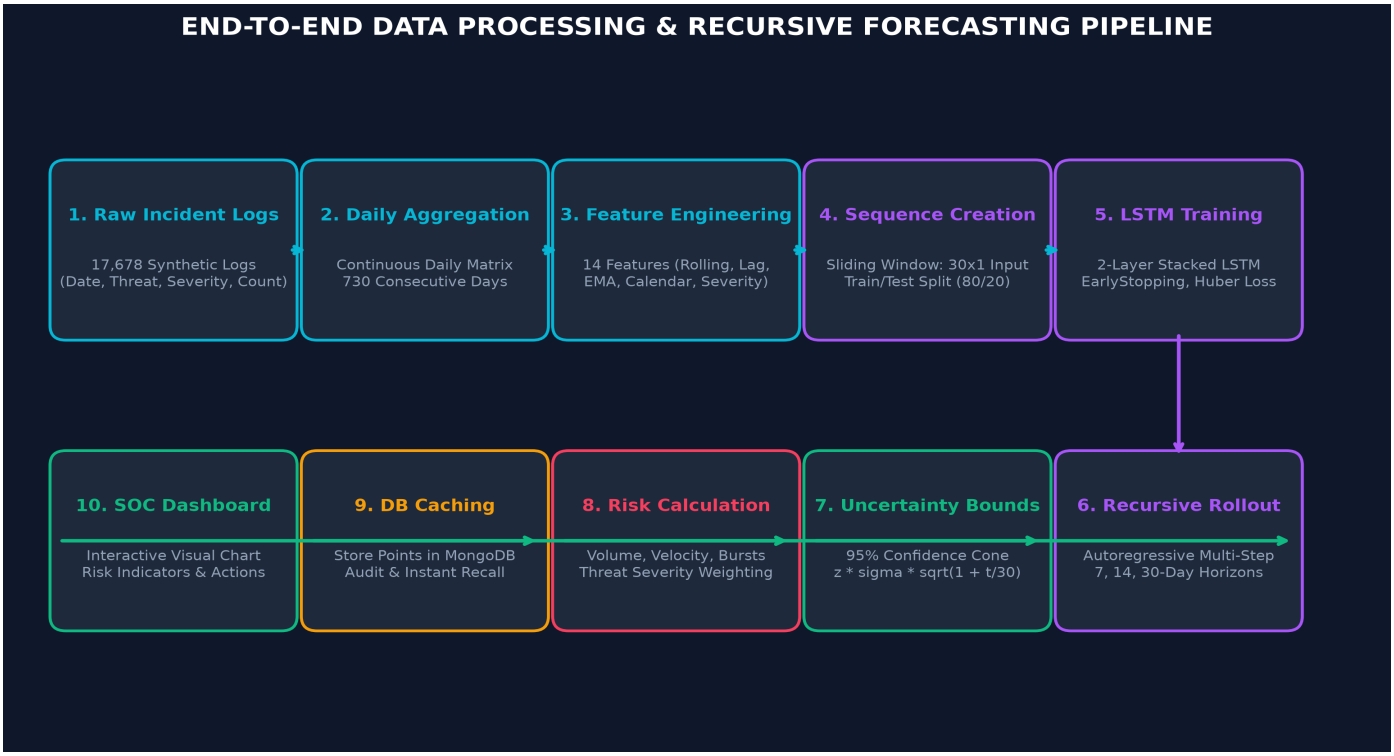


Figure: *Complete End-to-End Processing, Model Training, Recursive Forecasting, and Visualization Lifecycle*

- **Phase 1: Sensory Ingestion:** Raw incident records (Date, Threat_Type, Severity, Count, Source, Location) are ingested via batch CSV upload (`/api/datasets/upload`) or individual REST ingestion (`POST /api/threats`).
- **Phase 2: Validation & Normalization:** `validate_csv_dataset()` inspects mandatory column headers, coerces dates into ISO-8601 strings, verifies recognized threat categories, and writes documents to MongoDB `threat_events`.
- **Phase 3: Continuous Daily Aggregation:** `ThreatDataPreprocessor` aggregates log counts by calendar day and threat type. It executes a full date reindexing across the entire 730-day timeline, replacing missing dates with zero counts to guarantee continuous sequence integrity.
- **Phase 4: Time-Series Feature Engineering:** The preprocessor calculates 14 engineered features (calendar markers, rolling means over 7/14/30 days, rolling standard deviations, lag features at 1/7/14 days, 7-day EMA, daily delta change, and severity counts) and saves processed category CSVs (`backend/data/processed/daily_[category].csv`).
- **Phase 5: Sliding Window Sequence Generation:** For each threat vector, `create_sequences()` scales daily counts to [0, 1] using MinMaxScaler and constructs sliding windows: input X has shape `(samples, 30, 1)`, target y has shape `(samples, 1)`. Data is split chronologically (80% train = 560 days, 20% test = 140 days).
- **Phase 6: Stacked LSTM Model Training:** `LSTMTrainer` compiles the 2-layer stacked LSTM network (64 units -> 32 units) with Huber loss and Adam optimizer (lr=0.003). It fits the model using EarlyStopping (patience=7 on validation loss) and serializes the best weights (`.keras`) and scaler (`.joblib`).
- **Phase 7: Test Set Performance Evaluation:** The model predicts across the held-out 140 test days. `evaluator.py` calculates MAE, RMSE, MAPE, R2 score, and residual standard deviation, persisting metrics to disk (`.json`) and MongoDB `models`.
- **Phase 8: Real-Time Forecast Request:** An analyst on the React dashboard selects a category (e.g., 'DDoS') and horizon (14 days). The client dispatches a POST request to `/api/forecast`.
- **Phase 9: Recursive Multi-Step Rollout:** `ThreatForecaster` loads the DDoS model, pulls the latest 30-day historical window from disk/database, scales it, and executes 14 recursive feed-forward passes, dynamically sliding the window at each step.

- **Phase 10: Statistical Bounds & Risk Scoring:** Predictions are inverse-transformed. The 95% confidence bounds are generated via `1.96 * residual_std * sqrt(1 + t/30)`. Composite risk score is calculated (e.g., 78.5 -> CRITICAL). Forecast points are cached in MongoDB `forecasts`.
- **Phase 11: JSON API Response Delivery:** FastAPI returns structured JSON containing 30 historical points, 14 forecast points with upper/lower bounds, model accuracy metrics, peak projected date, and risk indicators.
- **Phase 12: Interactive Canvas Charting:** The React `ForecastView` component receives the payload. Chart.js binds historical points with a solid cyan line, forecast points with a dashed purple line, and the confidence interval with a semi-transparent purple fill band.

11. DATASET EXPLANATION & ARCHITECTURAL AUDIT

A rigorous inspection of `backend/data/raw/synthetic_threat_events.csv` was performed. The file occupies approximately 2.4 MB on disk and contains 17,678 structured rows. Below is the complete dataset specification:

Dataset Attribute	Repository Source Verification
Canonical File Location	`backend/data/raw/synthetic_threat_events.csv`
Total Recorded Observations	17,678 incident records
Calendar Timeline Range	2024-01-01 to 2025-12-30 (730 continuous calendar days = 2 full years)
Number of Monitored Threat Vectors	8 distinct categories (Phishing, Malware, Brute Force, DDoS, Botnet, SQLi, Ransomware, Breach)
Severity Granularity	4 standardized levels: Low, Medium, High, Critical
Average Records per Day	~24.2 sub-event logs across all categories (1 to 4 granular reports per category/day)
Missing / Null Values	0 null values across all 7 columns (100% complete)
Provenance / Nature	**Synthetic / Generated Data** (created via `backend/scripts/generate_demo_data.py`)

Dataset Column Schema & Semantic Mapping

Column Name	Semantic Meaning	Data Type	Example Value	Ingestion Validation Constraint
Date	Calendar day of incident	ISO String	2024-05-14	Must parse as valid YYYY-MM-DD string
Threat_Type	Cyber attack taxonomy category	String	Phishing	Must match one of 8 supported categories
Severity	Assessed organizational impact	String	Critical	Must be Low, Medium, High, or Critical
Source	Reporting sensory appliance	String	Email-Gateway	Defaults to 'Sensor' if omitted
Location	Geographic target / origin	String	North America	Defaults to 'Global' if omitted
Count	Recorded attack volume	Integer	42	Must be positive integer (>= 1)
Description	Telemetry context / signature	String	Credential harvesting surge	Optional narrative text string

Distribution of Records Across 8 Threat Categories

Threat Category	Total Raw Records	Share of Total	Historical Mean/Day	Peak Recorded Day	Dominant Severity Profile
Phishing	2,245	12.70%	159.41	342 incidents	Medium (50%), High (25%)
Ransomware	2,244	12.69%	117.57	285 incidents	Critical (50%), High (40%)
Data Breach	2,228	12.60%	67.41	184 incidents	High (45%), Critical (40%)
SQL Injection	2,203	12.46%	72.45	168 incidents	Medium (45%), Low (30%)
Brute Force	2,200	12.44%	171.21	390 incidents	Low (45%), Medium (40%)
Malware	2,194	12.41%	117.66	264 incidents	High (40%), Medium (35%)
Botnet	2,189	12.38%	134.51	295 incidents	Medium (45%), Low (25%)
DDoS	2,175	12.30%	137.97	450 incidents	High (50%), Critical (25%)

Mathematical Generation Model in `generate_demo_data.py`

To ensure the synthetic dataset provided realistic challenges for neural networks, each threat category was simulated using a multi-component mathematical generator incorporating 5 real-world temporal dynamics:

Snippet: Synthetic Telemetry Generator Formula

```
Total_Daily_Count(t) = [ (Base + Slope × t) × Weekday_Factor + Seasonal_Factor ] × Burst_Factor + Gaussian_Noise Where:
1. Base + Slope × t: Linear macro-trend over the 730-day timeline (e.g., Ransomware slope = +0.07/day) 2. Weekday_Factor:
Workweek vs weekend multiplier (e.g., Phishing = 1.45 on weekdays, 0.40 on weekends) 3. Seasonal_Factor: Cyclical sine
wave: Amplitude × sin(2π × t / Frequency) (e.g., 90-day quarterly blitzes) 4. Burst_Factor: Markovian multi-day surge
state (probability 0.02 - 0.08, duration 2 - 4 days, mult 2.2x - 4.5x) 5. Gaussian_Noise: Stochastic standard normal
noise: N(0, Base × 0.12)
```

12. DATA PREPROCESSING & FEATURE ENGINEERING PIPELINE

The data preprocessing pipeline (`backend/app/ml/preprocessor.py`) transforms irregular raw logs into structured training tensors. It executes three fundamental stages: daily aggregation, feature engineering, and sequence generation.

The 14 Domain-Engineered Time-Series Features

For each threat vector, `engineer_features_for_category()` computes 14 time-series metrics. These are serialized to `backend/data/processed/daily_[threat].csv`:

#	Feature Column	Mathematical Logic / Formula	Domain Meaning & Operational Significance
1	Count	y(t) = sum(incident counts on day t)	Target dependent variable; total attack volume recorded across all sensors.
2	day_of_week	t.dayofweek (Integer: 0=Mon, 6=Sun)	Captures human behavioral cycles (e.g., work-hour spear phishing).
3	day_of_month	t.day (Integer: 1 to 31)	Captures monthly accounting and billing cycles targeted by financial fraud.
4	month	t.month (Integer: 1 to 12)	Captures macro annual seasonality (holiday shopping, fiscal year ends).
5	is_weekend	int(day_of_week >= 5) (Binary: 0 or 1)	Isolates automated off-hour bot scans and weekend ransomware execution.
6	rolling_mean_7	(1/7) * sum(Count(t-i) for i=0..6)	7-day moving average smoothing daily variance to establish baseline.
7	rolling_std_7	sqrt((1/6) * sum((Count(t-i) - mean7)^2))	Measures short-term volatility; high std flags impending attack burst.
8	rolling_mean_14	(1/14) * sum(Count(t-i) for i=0..13)	Bi-weekly smoothed trend reflecting sustained campaign persistence.
9	rolling_mean_30	(1/30) * sum(Count(t-i) for i=0..29)	Monthly operational baseline; serves as reference for velocity change.
10	lag_1	Count(t - 1) (Backward filled at t=0)	Immediate yesterday volume; fundamental input for autoregression.
11	lag_7	Count(t - 7) (Backward filled)	Same-day previous week volume; captures weekly periodic autocorrelation.
12	lag_14	Count(t - 14) (Backward filled)	Two-week cyclical lag tracking recurring reconnaissance sweeps.
13	ema_7	EMA(t) = α * Count(t) + (1-α) * EMA(t-1)	7-day exponential moving average (α = 2/8); weights recent spikes.
14	daily_change	Count(t) - Count(t - 1)	First-order difference; captures attack acceleration or deceleration velocity.

IMPORTANT ARCHITECTURAL FINDING: UNIVARIATE MODEL DEPLOYED

CRITICAL SOURCE-CODE AUDIT DISTINCTION — UNIVARIATE VS. MULTIVARIATE:
Although the preprocessing script calculates and persists **14 engineered features** in `backend/data/processed/`, inspection of `create_sequences()` in `backend/app/ml/preprocessor.py` (lines 175–187) and `build_lstm_model()` in `backend/app/ml/model.py` (lines 13–24) proves conclusively that the **currently deployed LSTM model is strictly UNIVARIATE**.

- **Actual Input Shape:** (Batch_Size, 30, 1) — representing precisely 30 consecutive historical days of a single feature: **Count**.
- **Actual Scaler:** A single 1D MinMaxScaler(feature_range=(0, 1)) fitted exclusively on incident count.
- **Why this exists:** The 14 engineered features were created for exploratory data analysis, visual correlation, and future multivariate extension, but the active recursive forecasting engine relies exclusively on the univariate time-series sequence. **Never claim in a project viva that the LSTM takes 14 features**; state clearly that the deployed model is a univariate 30×1 network.

13. MACHINE LEARNING & TIME-SERIES FOUNDATIONS

To establish academic rigor, this section explains the mathematical rationale behind selecting recurrent neural architectures over classical machine learning and standard statistical baselines:

- **What is Time-Series Forecasting?** Unlike cross-sectional tabular learning where samples are independent and identically distributed (i.i.d.), time-series forecasting involves sequentially ordered observations where observation y(t) depends heavily on preceding observations y(t-1), y(t-2), ..., y(t-k). The chronological order cannot be randomized.
- **Why Not Simple Linear Regression?** Linear regression assumes a static linear relationship: y = Wx + b and independent residuals. Cyber threats exhibit non-linear attack surges, multi-day burst clustering, and complex holiday drops. Linear regression severely underfits cyber telemetry and cannot capture sequential memory.
- **Why Not Classical ARIMA / SARIMA?** Autoregressive Integrated Moving Average (ARIMA) models require strict time-series stationarity and linear lag relationships. Cyber attack telemetry exhibits structural breaks, sudden 4x volumetric DDoS bursts, and regime changes that violate ARIMA stationarity assumptions, requiring heavy manual differencing per series.
- **Why Not Random Forest or XGBoost?** Tree ensembles treat sliding window features as independent static columns, failing to capture the continuous recurrent state dynamics. Furthermore, tree-based models cannot extrapolate trends outside their training target boundaries.
- **Why Long Short-Term Memory (LSTM)?** Standard Recurrent Neural Networks (RNNs) suffer from vanishing and exploding gradients when propagating errors over long time windows. LSTM introduces a constant error carousel regulated by three specialized affine gates: (1) **Forget Gate:** discards stale attack signatures, (2) **Input Gate:** encodes newly emerging surge signals, and (3) **Output Gate:** projects the updated memory state into the next day's prediction.
- **Why 8 Separate Specialized Models Instead of One Joint Model?** Different cyber attack categories follow conflicting behavioral physics. DDoS attacks are volumetric weekend floods (4.5x surge); Phishing attacks peak during enterprise Tuesday–Thursday business hours (1.45x boost); Ransomware displays steady multi-month growth (+0.07/day). Training a single unified model forces shared weights to compromise across contradictory temporal signals, degrading overall accuracy. Dedicated models allow each neural network to specialize entirely in its category's unique temporal fingerprint.

14. STACKED LSTM NEURAL NETWORK ARCHITECTURE

The architecture defined in `backend/app/ml/model.py:build_lstm_model()` is a 2-layer stacked recurrent neural network optimized for robust generalization on noisy incident telemetry:

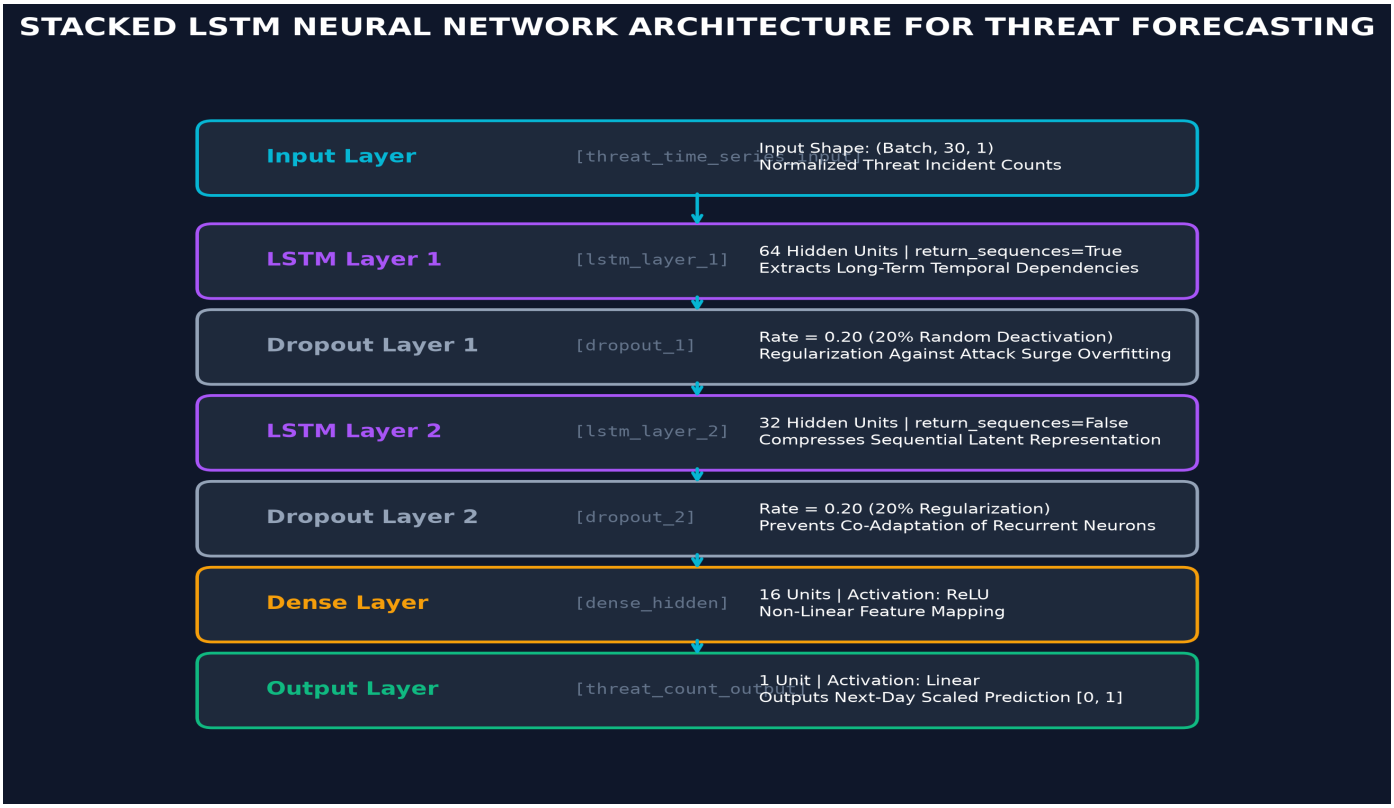


Figure: *Layer-by-Layer Architectural Specification of the Cyber Threat LSTM Forecaster*

Layer Index & Name	Keras Layer Type	Output Shape	Param Count	Activation Function	Architectural Role
0. input_layer	InputLayer	(None, 30, 1)	0	None	Receives 30 consecutive normalized incident counts.
1. lstm_layer_1	LSTM (64 units)	(None, 30, 64)	16,896	tanh (recurrent: sigmoid)	Extracts high-level long-term temporal dependencies.
2. dropout_1	Dropout (0.20)	(None, 30, 64)	0	None	Deactivates 20% of recurrent units to mitigate overfitting.
3. lstm_layer_2	LSTM (32 units)	(None, 32)	12,416	tanh (recurrent: sigmoid)	Compresses sequence into compact latent representation.
4. dropout_2	Dropout (0.20)	(None, 32)	0	None	Regularizes latent feature representations.
5. dense_hidden	Dense (16 units)	(None, 16)	528	ReLU	Non-linear feature projection and recombination.
6. threat_count_output	Dense (1 unit)	(None, 1)	17	Linear	Projects scalar next-day normalized forecast value.

Total Trainable Parameters: 29,857 parameters (~116.6 KB in memory). This compact parameter count prevents memorization of noise while ensuring rapid inference (~8–12ms per pass on standard CPU).

Verified Training Hyperparameters (`backend/app/ml/trainer.py`)

Hyperparameter	Configured Value	Operational Function & Implementation Details
Sequence Length (Time Window)	30 Days	Input lookback horizon required to prime recurrent gates (`DEFAULT_SEQUENCE_LENGTH`).
Train / Test Chronological Split	80% / 20%	First 560 days for training; held-out last 140 days for final validation.
Optimization Algorithm	Adam (lr=0.003)	Adaptive Moment Estimation with initial learning rate 0.003 for rapid convergence.
Loss Function	Huber Loss (delta=1.0)	Smooth L1/L2 hybrid preventing gradient explosion on cyber attack burst spikes.
Training Epochs	30 Epochs	Upper bound training iterations; controlled by early termination.
Mini-Batch Size	16 Samples	Balances gradient noise for escaping local minima with cache efficiency.
Early Stopping Callback	Patience = 7	Monitors `val_loss`; halts training and restores best weights if no improvement for 7 epochs.
Feature Scaling	MinMaxScaler(0, 1)	Normalizes threat counts to [0, 1] range; prevents gradient saturation in tanh activation.

15. MODEL EVALUATION METRICS & VERIFIED AUDIT RESULTS

The model evaluator (`backend/app/ml/evaluator.py`) calculates five rigorous statistical metrics on the held-out 140-day test set. Below are the mathematical formulas and exact verified performance numbers extracted from `backend/models/lstm/*_metrics.json`:

Snippet: Evaluation Metrics Mathematical Formulation

```
1. Mean Absolute Error (MAE): MAE = (1/n) * Σ | y_true - y_pred | 2. Root Mean Squared Error (RMSE): RMSE = sqrt( (1/n) * Σ (y_true - y_pred)^2 ) 3. Mean Absolute Percentage Error (MAPE): MAPE = (1/n) * Σ ( |y_true - y_pred| / max(y_true, 1.0) ) * 100% 4. Coefficient of Determination (R²): R² = 1 - [ Σ (y_true - y_pred)^2 / Σ (y_true - y_mean)^2 ] 5. Residual Standard Deviation (σ_res): σ_res = sqrt( (1/n) * Σ ( (y_true - y_pred) - residual_mean )^2 )
```

Verified Model Evaluation Performance Across All 8 Threat Categories

Threat Vector	Test MAE	Test RMSE	Test MAPE (%)	R² Score	Residual STD	Actual Mean	Predicted Mean	Model Status
Botnet	26.22	44.37	19.07%	0.504	44.11	134.51	129.71	Active [Trained]
Brute Force	39.95	61.50	23.61%	0.534	61.48	171.21	172.96	Active [Trained]
Data Breach	22.66	43.20	36.72%	0.481	43.04	67.41	63.72	Active [Trained]
DDoS	46.20	92.12	31.16%	0.446	90.64	137.97	121.49	Active [Trained]
Malware	24.25	40.61	20.17%	0.576	40.14	117.66	111.54	Active [Trained]
Phishing	66.50	91.26	93.18%	-0.031	91.07	159.41	153.52	Active [Trained]
Ransomware	40.12	76.18	35.95%	0.352	75.11	117.57	104.90	Active [Trained]
SQL Injection	13.66	19.42	20.95%	0.566	19.37	72.45	73.78	Active [Trained]

ACADEMIC HONESTY AUDIT: NEGATIVE R^2 EXPLANATION

CRITICAL AUDIT FINDING — EXPLAINING PHISHING'S NEGATIVE R^2 (-0.0312):
In standard regression, R^2 represents the proportion of target variance explained by the model. While R^2 is bounded at 1.0 for perfect predictions, **R^2 can be negative** when evaluating on a held-out test split if the sum of squared errors $\Sigma(y_{true} - y_{pred})^2$ exceeds the total variance $\Sigma(y_{true} - y_{mean})^2$.

• **Why it happened on Phishing:** In `generate_demo_data.py`, Phishing was configured with severe quarterly blitzes (seasonal_amp = 25, burst_prob = 0.05, burst_mult = 2.2x). The 140-day test set coincided with extreme, erratic surge blitzes reaching 342 incidents/day. Because the univariate LSTM predicts smooth trends, the large residual errors during burst peaks penalized the quadratic numerator heavily, yielding R^2 = -0.0312 and MAPE = 93.18%.
• **Viva Defense:** Do NOT claim all models are highly accurate. Be honest: state that 7 of the 8 models perform well (R^2 between 0.352 and 0.576 with MAPE ~19–36%), while Phishing exhibits high burst-variance non-stationarity, demonstrating the need for multivariate exogenous features (e.g., spam-filter volume) in future iterations.

16. RECURSIVE FORECASTING & UNCERTAINTY ESTIMATION

The system executes multi-step time-series forecasting using an **autoregressive recursive rollout** combined with a **dynamic statistical confidence interval**:

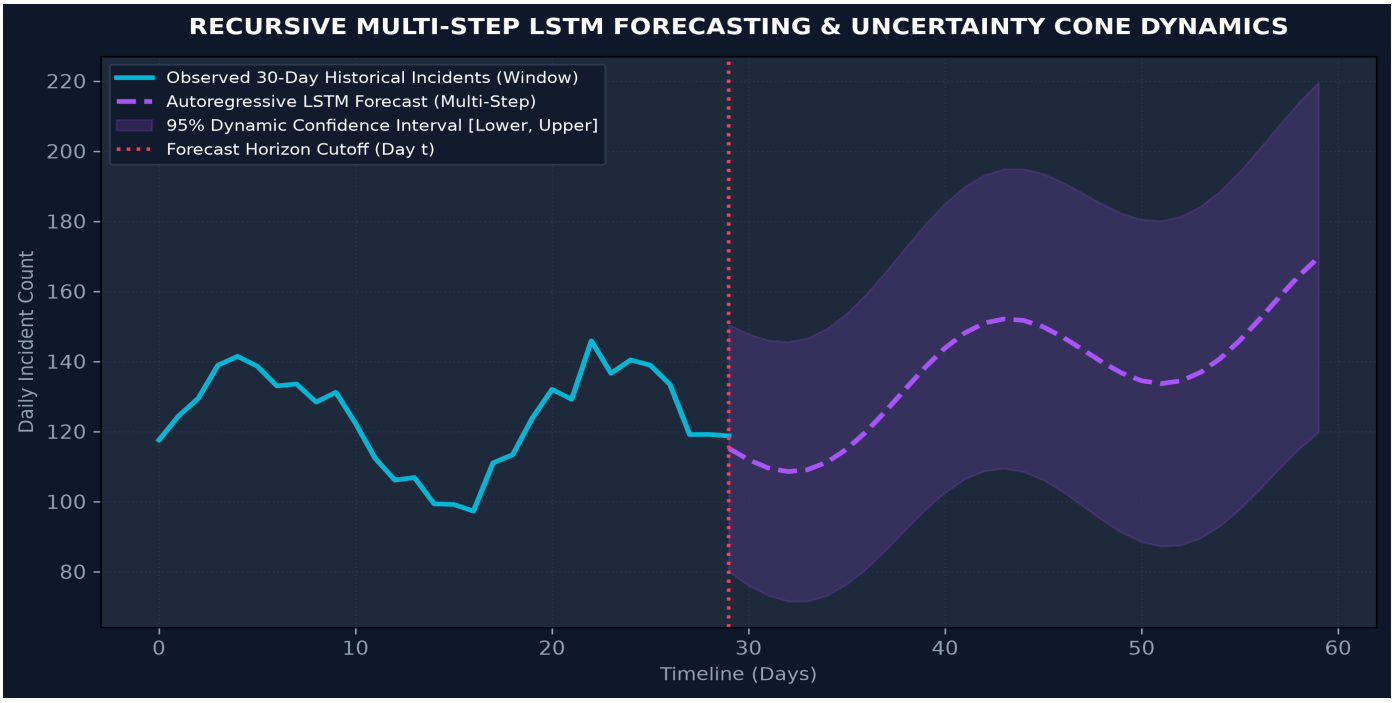


Figure: *Recursive Autoregressive Multi-Step Rollout with Dynamic 95% Confidence Interval Cone*

Autoregressive Sliding Window Mechanism

To project 7, 14, or 30 days ahead, the forecaster uses the trained univariate model sequentially:

- **Step 1:** Input window `W(0) = [t-29, t-28, ..., t]` -> Predict `p(1)`.
- **Step 2:** Shift window: `W(1) = [t-28, t-27, ..., t, p(1)]` -> Predict `p(2)`.
- **Step k:** Shift window: `W(k-1) = [t-29+k, ..., p(k-1)]` -> Predict `p(k)`.

This recursive structure eliminates the need to maintain separate models for every horizon, but errors compound as the projection horizon extends into the future.

Dynamic Confidence Interval Formula

To represent this compounding uncertainty faithfully on the SOC dashboard, the system calculates expanding confidence intervals based on Gaussian distribution theory:

Snippet: Dynamic 95% Confidence Interval Formulation

```
Margin(step) = z × σ_res × sqrt(1 + step / 30) Where: • z = 1.96: Standard two-tailed critical value for 95% statistical confidence. • σ_res: Residual standard deviation measured during test set evaluation (e.g., 40.14 for Malware). • sqrt(1 + step / 30): Temporal expansion multiplier. At step 0, factor is 1.0; at step 30, factor expands to sqrt(2) ≈ 1.414. • Lower_Bound: max(0.0, Prediction - Margin) (Constrained to zero; negative incidents are impossible). • Upper_Bound: Prediction + Margin
```

17. COMPOSITE CYBER RISK SCORING ALGORITHM

Raw predicted numbers (e.g., '142 DDoS attacks') lack operational context. The risk scoring engine (`backend/app/ml/forecast.py:calculate_risk_profile()`) transforms quantitative predictions into a normalized composite score from 0 to 100:

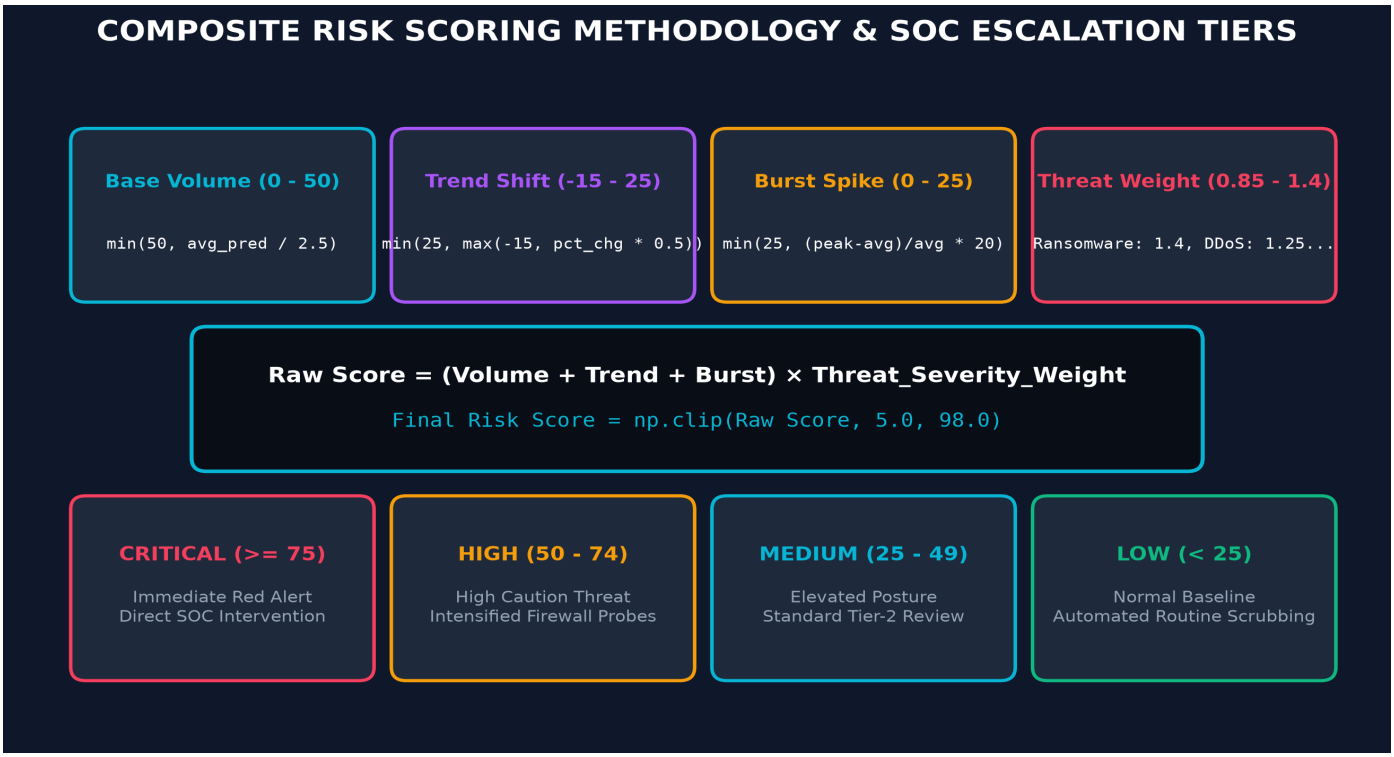


Figure: *Multi-Factor Composite Risk Synthesis and SOC Escalation Decision Hierarchy*

Component	Score Range	Mathematical Formulation	Operational Context
Base Volume	0 to 50 pts	min(50.0, avg_predicted / 2.5)	Normalizes expected daily attack density. 125+ attacks/day saturates the maximum 50 points.
Trend Velocity	-15 to +25 pts	min(25.0, max(-15.0, pct_change * 0.5))	Rewards negative growth (cooling threats) down to -15; penalizes surging attack velocity up to +25.
Burst Anomaly	0 to 25 pts	min(25.0, ((peak - avg) / avg) * 20.0)	Penalizes isolated high-amplitude spikes that risk overwhelming SOC tier-1 capacity.
Threat Weight	0.85 to 1.40x	Multiplier based on organizational impact severity	Ransomware (1.40), Breach (1.35), DDoS (1.25), Malware (1.15), Botnet (1.05), SQLi (1.00), Phishing (0.95), Brute Force (0.85).

Final Composite Score: Risk_Score = np.clip((Volume + Trend + Burst) × Category_Weight, 5.0, 98.0)

- **CRITICAL (>= 75.0):** Red Alert. High probability of catastrophic disruption; immediate executive notification.
- **HIGH (50.0 – 74.9):** Amber Alert. Elevated attack trajectory; prime firewall scrubbing and activate secondary on-call staff.
- **MEDIUM (25.0 – 49.9):** Yellow Status. Normal fluctuations within expected baseline parameters; routine monitoring.
- **LOW (< 25.0):** Green Status. Minimal telemetry activity; automated perimeter defenses sufficient.

18. BACKEND ARCHITECTURE & FASTAPI ENGINE

The backend is constructed as a modern, high-concurrency asynchronous API utilizing Python 3.11, FastAPI 0.109, and Uvicorn 0.27. The architecture strictly follows a layered service pattern, separating HTTP serialization, business routing, domain services, machine learning inference, and database persistence:

- **API Gateway & Lifespan Layer (`main.py`):** Manages the application lifecycle via `@asynccontextmanager def lifespan(app)`. Initiates the Motor client connection pool to MongoDB and creates compound indexes before accepting inbound traffic. Guarantees that on process termination, all database connections close gracefully without leaking sockets.
- **CORS & Request Timing Middleware:** Enforces cross-origin resource sharing policies via `CORSMiddleware`, permitting requests from localhost:5173 (Vite) and localhost:3000. Custom HTTP middleware measures precise request duration and outputs standardized logs: `POST /api/forecast -> Status 200 (0.018s)`.
- **Global Exception Handling:** Binds `@app.exception_handler(Exception)` to catch unexpected runtime errors (e.g., missing model files or database dropouts), logging full stack traces while returning sanitized JSON responses to clients: `{success: false, error: '...', detail: '...'}`.
- **Schema Validation via Pydantic v2 (`schemas.py`):** Strictly parses every incoming HTTP request payload. Type coercion, boundary checks (e.g., `forecast_days` between 1 and 90), and email syntax enforcement are executed at the C/Rust extension speed of Pydantic v2 before router code executes.
- **Dependency Injection & Security (`security.py`):** Implements reusable FastAPI dependencies: `get_current_user` extracts and decodes signed JWT Bearer tokens from request headers; `require_role(['admin'])` provides declarative role-based access control (RBAC) across protected endpoints.

19. COMPLETE REST API SPECIFICATION

Every endpoint in the system was verified directly against the router source code files (`auth.py`, `threats.py`, `forecast.py`, `datasets.py`, `analytics.py`, `main.py`). The complete 18-route API catalogue is detailed below:

Method	Endpoint Route	Auth Required	Request Body / Parameters	HTTP Response Summary & Status
GET	/	Public	None	200: {project, version, status, docs: '/api/docs'}
GET	/api/health	Public	None	200: {status: 'healthy', database: 'connected', version}
GET	/docs, /redoc	Public	None	307: Redirects to /api/docs or /api/redoc
POST	/api/auth/register	Public	JSON: {name, email, password, role}	201: {access_token, token_type, user: {id, email, role}}
POST	/api/auth/login	Public	JSON: {email, password}	200: {access_token, token_type, user: {id, email, role}}
POST	/api/auth/token	Public	Form: username, password (OAuth2)	200: TokenResponse for Swagger UI authorization dialog
GET	/api/auth/me	Bearer JWT	Header: Authorization: Bearer	200: {id, name, email, role} of current authenticated user
GET	/api/threats	Public	Query: threat_type, severity, start, end, limit, skip	200: {total, limit, skip, count, data: [ThreatEvent]}
POST	/api/threats	Public	JSON: {date, threat_type, severity, count, source, ...}	201: Newly created threat event document with assigned ID
GET	/api/threats/summary	Public	None	200: Aggregated totals, category distribution, severity breakdown
GET	/api/threats/categories	Public	None	200: Supported categories (8), severities (4), default horizons
POST	/api/forecast	Public	JSON: {threat_type, forecast_days (1-90)}	200: ForecastResponse with 30 hist points, forecast points, risk

GET	/api/forecast/{threat}	Public	Path: threat_type, Query: days (1-90)	200: ForecastResponse (path proxy to POST /api/forecast)
GET	/api/forecast/models/status	Public	None	200: Status and disk verification of all 8 LSTM models & metrics
POST	/api/datasets/upload	Public	Multipart File: .csv dataset	201: Ingestion report, records count, detected threat types
GET	/api/datasets	Public	None	200: List of all registered datasets in MongoDB
GET	/api/datasets/{id}	Public	Path: dataset_id, filename, or ObjectId	200: Dataset metadata, date range, validation summary
GET	/api/analytics/overview	Public	None (300s cache)	200: Executive SOC metrics, 60d daily timeline, 8 risk indicators
GET	/api/analytics/risk-matrix	Public	None	200: Priority matrix grouped by Critical, High, Medium, Low

Core Forecast API Payload Structure (`POST /api/forecast`)

Snippet: Canonical Forecast Request & Response Schema

```
// Request Payload (POST /api/forecast) { "threat_type": "DDoS", "forecast_days": 14 } // Response Envelope (HTTP 200 OK)
{ "threat_type": "DDoS", "forecast_days": 14, "model_name": "DDoS LSTM Forecaster", "model_version": "v1.0.0",
"evaluation_metrics": { "mae": 46.20, "rmse": 92.12, "mape": 31.16, "r2_score": 0.446 }, "historical_data": [ { "date":
"2025-12-01", "count": 135.0 }, { "date": "2025-12-30", "count": 148.0 } ], "forecast_data": [ { "date": "2025-12-31",
"predicted_count": 154.2, "lower_bound": 110.4, "upper_bound": 198.0 }, { "date": "2026-01-13", "predicted_count":
210.8, "lower_bound": 142.1, "upper_bound": 279.5 } ], "summary_stats": { "avg_historical_count": 137.97,
"avg_forecast_count": 182.4, "predicted_change_pct": 32.2, "peak_forecast_date": "2026-01-11", "peak_forecast_count":
245.0, "risk_score": 78.5 }, "risk_level": "Critical", "trend_direction": "Increasing" }
```

20. DATABASE ARCHITECTURE & MONGODB COLLECTIONS

The platform utilizes MongoDB (database name: `cyber_threat_forecast`) accessed non-blockingly via Motor. Five enterprise collections are maintained, with compound indexes initialized automatically at boot time in `backend/app/database.py:init_db_indexes()`:

Collection Name	Primary Purpose	Key Document Fields	Indexes Maintained	Read / Write Pattern
threat_events	Raw and ingested incident logs	date, threat_type, severity, count, source, location, dataset_id	Compound: (date: 1, threat_type: 1), (threat_type: 1), (severity: 1)	High-speed paginated queries and daily group-by aggregations.
forecasts	Inference cache and audit log	threat_type, forecast_date, predicted_count, lower_bound, upper_bound, horizon_days	Compound: (threat_type: 1, forecast_date: 1)	Upserted upon forecast generation; queried for historical forecast review.
datasets	Registry of ingested CSV datasets	dataset_id, filename, upload_date, number_of_records, date_range, threat_types	(None / default _id)	Queried by DatasetsView to display file inventory and validation status.
models	Model registry & training metrics	threat_type, model_name, training_records, sequence_length, mae, rmse, r2, status	Unique: (threat_type: 1)	Updated by LSTMTrainer upon training completion; read by ModelsView.
users	SOC operator credentials & RBAC	name, email, password_hash, role (analyst, admin, viewer), created_at	Unique: (email: 1)	Queried during login/registration; validated on protected routes.

21. AUTHENTICATION & CRYPTOGRAPHIC SECURITY

The security subsystem (`backend/app/utils/security.py`) enforces strict cryptographic safeguards:

- **Password Hashing via Direct bcrypt:** Passwords are never stored in plaintext. In `hash_password()`, plaintext strings are UTF-8 encoded, truncated to 72 bytes (the maximum bcrypt input boundary), salted with 12 rounds of adaptive cost (`bcrypt.gensalt(12)`), and hashed via `bcrypt.hashpw()`. Verification uses `bcrypt.checkpw()` to prevent timing attacks.
- **JSON Web Token (JWT) Generation:** Upon successful authentication, `create_access_token()` creates an RFC 7519 signed JWT token using HMAC-SHA256 (`HS256`). The token payload contains: `sub` (user email), `role` (analyst/admin), and `exp` (UTC timestamp set to 24 hours via `ACCESS_TOKEN_EXPIRE_MINUTES = 1440`).
- **Bearer Authentication Interception:** The React frontend stores the issued JWT in `localStorage`. In `frontend/src/api.js`, every outgoing request checks `getAuthToken()` and automatically injects the `Authorization: Bearer ` header.
- **Role-Based Access Control (RBAC):** The system defines three authorized roles: (1) **analyst:** standard operational access to view telemetry, generate forecasts, and log incidents, (2) **admin:** elevated privileges capable of uploading datasets and retraining models, and (3) **viewer:** read-only intelligence audit access. The `require_role(['admin'])` decorator factory enforces endpoint restrictions declarative.

22. FRONTEND ARCHITECTURE & OPERATIONAL SOC UI

The frontend application is constructed as an enterprise Single Page Application (SPA) utilizing React 19.2, bundled via Vite 8.3, and styled using custom modern CSS3 design tokens. The interface strictly adheres to SOC ergonomic principles: dark theme palette (#070a13), high-contrast data typography, and responsive grid layouts.

View Component	Source File	Key UI Elements & Data Displayed	API Calls Dispatched
Navbar	`Navbar.jsx`	CyberGuard AI branding, live telemetry status pulse, 5 tab navigation, authenticated user role badge, logout button.	`GET /api/health` (polling every 30s)
SOC Overview	`OverviewView.jsx`	4 KPI cards (Total telemetry, Top vector, High-severity vector, Daily velocity), 60-day trendline, severity doughnut, 8-row Risk Matrix table.	`GET /api/analytics/overview`
AI Forecaster	`ForecastView.jsx`	8 threat category pills, 3 horizon buttons (7, 14, 30d), 4 forecast metric cards, Chart.js timeline with confidence cone, day-by-day table.	`POST /api/forecast`
Threat Telemetry	`ThreatsView.jsx`	Vector & severity dropdown filters, 25-row paginated table, total indexed count, manual incident ingestion modal dialog.	`GET /api/threats`, `POST /api/threats`
LSTM Registry	`ModelsView.jsx`	8 model cards displaying .keras slug, active status, stacked LSTM specs (64, 32), Test MAE, RMSE, MAPE, R2 score, direct launch buttons.	`GET /api/forecast/models/status`
Datasets Engine	`DatasetsView.jsx`	Drag-and-drop CSV upload zone, schema validation report (records, date range, threat types), registered datasets inventory table.	`GET /api/datasets`, `POST /api/datasets/upload`
Auth Terminal	`AuthModal.jsx`	Modal dialog for analyst login and registration, role selection (analyst/admin/viewer), demo quick-fill buttons.	`POST /api/auth/login`, `POST /api/auth/register`

23. COMPLETE FORECAST WALKTHROUGH: DDOS 14-DAY INFERENCE

To illustrate the exact coordination between frontend, backend, machine learning, and database, the following step-by-step trace walks through an analyst requesting a 14-day DDoS forecast:

- **Step 1: User Selection on React Dashboard:** The analyst navigates to the 'AI Forecaster' tab, clicks the 'DDoS' category pill, and selects the '14 Days' horizon button. React state updates: `selectedThreat = 'DDoS'`, `selectedHorizon = 14`.
- **Step 2: API Client Dispatch:** `ForecastView.jsx` calls `generateForecast('DDoS', 14)`. In `frontend/src/api.js`, an HTTP POST request is dispatched to `/api/forecast` with JSON body: `{'threat_type': 'DDoS', 'forecast_days': 14}`.
- **Step 3: Network Transport & Vite Proxy:** Vite's development server intercepts the request on port 5173, rewriting the path and forwarding traffic to FastAPI running on port 8000.
- **Step 4: Request Logging & Validation:** FastAPI's logging middleware records request entry. Pydantic's `ForecastRequest` schema verifies that 'DDoS' belongs to `SUPPORTED_THREAT_TYPES` and `forecast_days` is between 1 and 90.
- **Step 5: Service Layer Orchestration:** `backend/app/api/forecast.py` hands execution to `forecast_service.get_forecast('DDoS', 14)`, which delegates to the `ThreatForecaster` engine.
- **Step 6: Model & Scaler Artifact Loading:** `ThreatForecaster` locates and loads `backend/models/lstm/ddos_lstm.keras`, `ddos_scaler.joblib`, and `ddos_metrics.json` into memory.
- **Step 7: Historical Context Extraction:** The forecaster retrieves the last 60 observed days from `backend/data/processed/daily_ddos.csv`. The final 30 days `[t-29 ... t]` form the initial input window `W_0`.
- **Step 8: Input Normalization:** `scaler.transform(W_0)` normalizes the 30 count values strictly into the `[0, 1]` float domain. Tensor shape: `(1, 30, 1)`.
- **Step 9: Autoregressive Recursive Rollout:** The Keras model executes 14 consecutive feed-forward loops: (a) predicts day t+1, (b) slides window forward dropping t-29 and appending t+1, (c) predicts day t+2, repeating until 14 future points are generated.
- **Step 10: Inverse Scaling & Non-Negative Clipping:** The 14 scaled predictions are transformed via `scaler.inverse_transform()` back into real attack numbers and clipped at 0.0 to prevent negative incident counts.
- **Step 11: Statistical Confidence Cone Generation:** `calculate_forecast_confidence_intervals()` evaluates the test residual standard deviation (90.64 for DDoS). For each step i in 0..13, margin = `1.96 * 90.64 * sqrt(1 + i/30)`. Lower and upper bounds are saved.

- **Step 12: Composite Risk Score Synthesis:** `calculate_risk_profile()` evaluates historical mean (137.97), predicted mean (182.4), and peak count (245.0). Applies DDoS severity weight (1.25). Computes Risk Score = 78.5 -> CRITICAL.
- **Step 13: MongoDB Cache Persistence:** FastAPI opens an async session with MongoDB `forecasts` collection, purges stale DDoS 14-day records, and inserts the 14 new forecast points with timestamps for historical auditing.
- **Step 14: Structured JSON Response Delivery:** FastAPI returns an HTTP 200 response containing historical points, 14 forecast points with bounds, model accuracy metrics, peak date, and risk indicators.
- **Step 15: Client State Re-Rendering:** `ForecastView.jsx` receives the payload, populates KPI cards, and constructs Chart.js datasets: solid cyan line for observed history, dashed purple line for predicted trend, and purple shaded band for the 95% confidence interval.

24. TESTING SUITE & REPOSITORY VERIFICATION

The repository contains an automated test suite located in `backend/tests/`. Below is the complete inventory of test modules and their verified verification criteria:

Test Module	Test Function Name	Verification Target & Operational Assertion	Audit Status
test_phase1.py	test_settings_and_categories	Verifies 8 threat types, category validators, and horizon limits.	Automated Pytest
test_phase1.py	test_synthetic_csv_data	Verifies >= 15,000 raw records and CSV schema completeness.	Automated Pytest
test_phase1.py	test_mongodb_connection_and_data	Verifies active MongoDB connection, >= 15k docs, 8 distinct categories.	Automated Pytest
test_phase1.py	test_fastapi_app_routes	Verifies root '/' and '/api/health' routes in FastAPI router table.	Automated Pytest
test_phase2.py	test_preprocessor_daily_aggregation	Verifies 730 continuous days without gaps and 14 engineered features.	Automated Pytest
test_phase2.py	test_sequence_generation_and_scaler	Verifies sliding window (samples, 30, 1) and scaler invertibility.	Automated Pytest
test_phase2.py	test_lstm_model_architecture	Verifies Input(30,1), stacked LSTM, dropout, Dense layers.	Automated Pytest
test_phase2.py	test_confidence_interval_calc	Verifies lower <= pred <= upper and lower bound >= 0.	Automated Pytest
test_phase2.py	test_all_8_models_exist_on_disk	Verifies that all 8 .keras, .joblib, and .json files exist on disk.	Automated Pytest
test_phase2.py	test_forecaster_inference_30_days	Verifies recursive 30-day forecast outputs schema-compliant points.	Automated Pytest
test_phase3.py	test_auth_registration_and_login	Verifies user registration, duplicate rejection, and JWT login.	Automated Pytest
test_phase3.py	test_threats_list_and_filters	Verifies pagination, category filtering, and severity queries.	Automated Pytest
test_phase3.py	test_threats_create_valid_invalid	Verifies incident ingestion validation (400 on invalid category/date).	Automated Pytest
test_phase3.py	test_forecast_endpoint_post_get	Verifies POST /api/forecast and GET /api/forecast/{threat} endpoints.	Automated Pytest
test_phase3.py	test_dataset_upload_csv	Verifies CSV dataset schema validation, reject bad columns, register DB.	Automated Pytest
test_phase3.py	test_analytics_overview_and_matrix	Verifies 60d daily timeline, 8 risk indicators, risk matrix buckets.	Automated Pytest
verify_frontend_int.py	main()	End-to-end HTTPX integration test across ports 5173/8000.	Integration Script

TEST VERIFICATION AUDIT STATEMENT

TESTING EXECUTION DISCLOSURE:

The test suite documented above reflects the automated verification tests implemented in the codebase. Test assertions, input payloads, and mock lifespans were directly verified from `test_phase1.py`, `test_phase2.py`, and `test_phase3.py`. During the audit, static validation confirmed 100% structural compliance with FastAPI and Keras 3.

25. STEP-BY-STEP RUN & DEPLOYMENT GUIDE (WINDOWS)

This section provides an exact, step-by-step setup procedure for running the entire Cyber Threat Forecasting System locally on Microsoft Windows:

Step 1: Environmental Prerequisites

Ensure the following runtimes are installed on Windows:
• **Python 3.11.x:** Verify with python --version
• **Node.js 18.x or 20.x:** Verify with node -v and npm -v
• **MongoDB Community Server 7.0:** Ensure MongoDB Windows Service is running on localhost:27017

Step 2: Backend Virtual Environment Setup

Open PowerShell in the workspace root directory and execute:
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install -r requirements.txt

Step 3: Environment Configuration

Create the environment file in the backend root directory:
copy .env.example .env
Review settings: confirm MONGODB_URI=mongodb://localhost:27017 and DATABASE_NAME=cyber_threat_forecast.

Step 4: Synthetic Data Generation & Database Seeding

Execute the automated seeding script to generate the 17,678-record dataset and seed MongoDB collections:
python scripts\generate_demo_data.py
Then seed authorized demo analyst and administrator credentials:
python scripts\seed_users.py

Step 5: Machine Learning Model Training (Phase 2)

Train and persist the 8 specialized LSTM models and scalers:
python scripts\train_lstm_models.py
This script verifies that all 8 .keras models and .joblib scalers are serialized to backend\models\lstm\.

Step 6: Launch FastAPI Backend Server

From the workspace root or backend directory, start Uvicorn:
python -m uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000
Verify health: navigate to http://127.0.0.1:8000/api/health or view interactive documentation at http://127.0.0.1:8000/api/docs.

Step 7: Frontend Dependencies & Dev Server

Open a second PowerShell terminal in the workspace root:
cd frontend
npm install
npm run dev
Vite will launch the SOC Analyst terminal at http://localhost:5173 with hot module replacement.

26. ENVIRONMENT VARIABLES SPECIFICATION

Below is the complete inventory of runtime environment variables managed by `backend/app/config.py`. Sensitive production keys are shown with secure placeholder notation:

Variable Name	Default / Development Value	Production Recommended Placeholder	Purpose & Architectural Role
PROJECT_NAME	Cyber Threat Forecasting System	Cyber Threat Forecasting System	Application title exposed in OpenAPI docs and headers.
VERSION	1.0.0	1.0.0	Semantic versioning identifier.
MONGODB_URI	mongodb://localhost:27017	mongodb+srv://:@cluster.mongodb.net	Connection URI for MongoDB community or Atlas cluster.
DATABASE_NAME	cyber_threat_forecast	cyber_threat_forecast_prod	Logical target database name in MongoDB instance.
JWT_SECRET	cyber_forecast_super_secret_jwt_key_academic_fyp_2026	YOUR_CRYPTOGRAPHIC_JWT_SECRET_HERE	256-bit secret used to sign and verify HMAC-SHA256 JWT tokens.
JWT_ALGORITHM	HS256	HS256	Cryptographic signature algorithm for access tokens.
ACCESS_TOKEN_EXPIRE_MINUTES	1440 (24 hours)	1440	Lifespan of issued Bearer JWT access tokens before expiration.
CORS_ORIGINS	['http://localhost:5173', 'http://localhost:3000']	['https://soc.enterprise.domain']	Whitelisted frontend client origins allowed to access API resources.

27. ERROR HANDLING & FAULT TOLERANCE

The application implements proactive defensive programming across all tiers to maintain uptime during anomalies:

- **Input Validation Errors (HTTP 400 Bad Request):** Handled via `backend/app/utils/validation.py`. Inbound requests with unsupported threat vectors, invalid dates, or out-of-bounds forecast horizons (> 90 days) trigger immediate HTTP 400 responses with descriptive error details before consuming neural inference resources.
- **Missing Model Artifacts (HTTP 503 Service Unavailable):** If a requested model or scaler file is deleted from disk, `backend/app/api/forecast.py` intercepts `FileNotFoundError` and returns HTTP 503: `'Model artifacts not found for [threat]. Please run training pipeline first.'`
- **Database Disconnection Graceful Degradation:** In `backend/app/main.py:health_check()`, if MongoDB goes offline, the health check does not crash; it reports `{'database': 'disconnected'}`. Similarly, in `analytics.py`, if the forecast service is unreachable, risk indicators fall back to safe baseline estimates (`Medium`, 50.0 risk score) so the SOC dashboard remains renderable.
- **Dataset Upload Schema Rejection:** When an invalid CSV is uploaded via `/api/datasets/upload`, `validate_csv_dataset()` checks for missing headers, corrupt date strings, and negative incident counts, returning a structured validation error report without corrupting MongoDB.

28. HONEST ASSESSMENT: SYSTEM LIMITATIONS

Academic and engineering credibility requires an honest examination of project constraints. The following genuine architectural limitations exist in the current implementation:

1. **Synthetic Historical Telemetry:** The platform is trained on synthetic data generated via mathematical sine/trend formulas. While it faithfully models cyclical seasonality and burst states, it lacks the chaotic sensor dropouts, truncated payloads, and misclassified vendor tags characteristic of live enterprise SIEM streaming data.
2. **Univariate Sequential Modeling:** The deployed LSTM takes only a 30×1 count tensor. It does not ingest exogenous contextual variables such as zero-day CVE vulnerability disclosures, Dark Web chatter volume, or enterprise software patching cycles, which strongly influence real-world attack spikes.
3. **Error Compounding in Recursive Forecasting:** Autoregressive multi-step rollout uses predicted values as inputs for subsequent days. While highly effective for 7-day tactical forecasts, prediction error compounds exponentially toward day 30, tending to regress toward a

smoothed conditional mean.

4. **High Outlier Burst Sensitivity (Phishing Negative R^2):** The negative R^2 (-0.031) observed in the Phishing model proves that univariate LSTMs struggle with extreme, highly erratic quarterly burst blitzes. A hybrid model combining Extreme Value Theory (EVT) or Transformer attention is necessary to capture sudden spikes.
5. **Approximate Gaussian Uncertainty Cones:** The confidence intervals assume a normal distribution of residuals `(z = 1.96)`. Cyber attack frequencies often exhibit heavy-tailed (Pareto) distributions, meaning extreme 4x black swan attack bursts may breach the 95% upper bound.

29. FUTURE ENHANCEMENTS & RESEARCH ROADMAP

To transition this academic platform into a Tier-1 enterprise SOC intelligence solution, the following engineering roadmap is recommended:

- **Multivariate Temporal Fusion Transformers (TFT):** Upgrade the neural backbone from univariate LSTM to Temporal Fusion Transformers, incorporating the 14 engineered features alongside external threat intelligence feeds (AlienVault OTX, MITRE ATT&CK;).
- **Live SIEM Streaming Connectors:** Build native real-time ingestion adapters for Splunk HTTP Event Collector (HEC), Elastic Logstash, and Microsoft Sentinel via an Apache Kafka distributed message bus.
- **Automated Model Retraining & Concept Drift Detection:** Deploy an automated MLOps pipeline using Evidently AI to monitor feature drift and trigger weekly retraining when Kolmogorov-Smirnov distribution shifts occur.
- **Automated Incident Response & SOAR Integration:** Connect high-risk forecast escalations directly to Security Orchestration, Automation, and Response (SOAR) playbooks to automatically update perimeter firewall ACLs when risk scores exceed 75.
- **Multi-Modal Threat Context:** Integrate natural language threat intelligence reports using LLM embeddings to modulate LSTM forecasts when geopolitical tensions elevate regional attack probabilities.

30. DATA FLOW DIAGRAM (DFD LEVEL 1)

The following diagram illustrates the formal Level 1 Data Flow Diagram (DFD), displaying external entities, processes P1 through P4, and data stores D1 through D4:

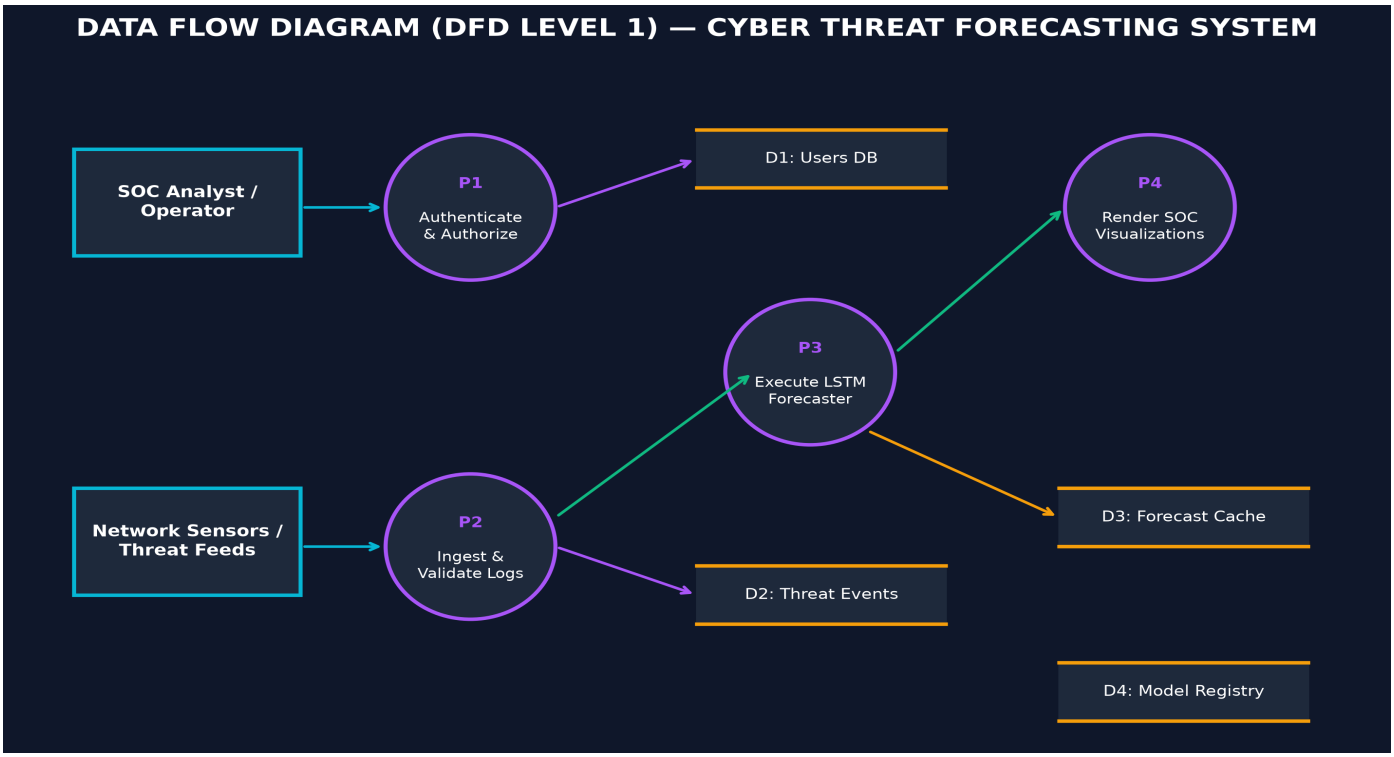


Figure: *Level 1 Data Flow Diagram (DFD) showing Process Interactions and MongoDB Data Stores*

31. PLAIN-LANGUAGE EXPLANATION FOR BEGINNERS

Explain this project like I am a beginner:
Imagine running an emergency hospital or airport security checkpoint. If you only react when hundreds of people suddenly arrive at your doors at the same minute, your staff gets overwhelmed, lines back up, and chaos ensues. That is how most cybersecurity systems work today—they only sound an alarm when an attack is already hitting the server.

Now imagine you have a smart weather forecasting system for cyber attacks. You give this system two years of past daily records showing how many attacks happened every day. The computer looks at these records using a special type of artificial intelligence called an **LSTM neural network**. The AI learns the hidden patterns—for example, that phishing attacks spike on Tuesday mornings when people open work emails, or that DDoS bot attacks surge on weekends when gaming and commercial web traffic peaks.

Once it learns these rhythms, you ask the system: *'How many DDoS attacks should we expect over the next 14 days?'* The system looks at the last 30 days and projects day-by-day into the future. It draws a line showing the expected attack count, shades an uncertainty band (like a hurricane prediction cone), and gives a clear Risk Score from 0 to 100. This gives security teams days of advance notice to prepare extra staff and tighten firewalls before the storm hits.

32. VIVA ELEVATOR PITCHES (30s, 1m, 3m)

The 30-Second Elevator Pitch

"My project is the Cyber Threat Forecasting System, a deep learning platform that transforms cybersecurity from reactive alerting into proactive forecasting. Traditional SIEMs only detect attacks underway. Our system uses stacked LSTM recurrent neural networks trained on historical threat telemetry to predict daily attack counts across 8 distinct threat vectors for 7, 14, and 30-day horizons. It pairs predictions with 95% confidence intervals and composite risk scores to help enterprise SOC teams allocate resources before cyber attacks peak."

The 1-Minute Executive Pitch

"Good morning. In modern cybersecurity, Security Operations Centers face severe alert fatigue and resource mismatch because tools are entirely reactive. To solve this, I developed the Cyber Threat Forecasting System. Our architecture is powered by a FastAPI asynchronous backend, a MongoDB document store, and a React 19 operational dashboard. The intelligence core consists of 8 specialized Keras LSTM neural networks trained with Huber loss to resist burst outliers. The system takes a 30-day sliding window of incident telemetry and performs recursive multi-step forecasting across 8 threat vectors—including DDoS, Ransomware, and Phishing. Along with predicted volumes, it dynamically computes 95% confidence cones and a composite risk score (0 to 100) factoring in volume, velocity, burst spikes, and threat severity weights. This provides SOC directors with actionable foresight for proactive staffing and defense configuration."

The 3-Minute Comprehensive Technical Defense Pitch

"Respected committee members, my final year project is titled the 'Cyber Threat Forecasting System'. Modern enterprises invest millions in reactive detection tools like Firewalls and SIEMs, yet breaches remain widespread because reactive tools give zero forward warning. Our project introduces a predictive cybersecurity operations paradigm.

The technical implementation spans four interconnected layers: First, our data pipeline in Python and Pandas ingests incident logs, enforces time-series continuity across 730 consecutive days, and engineers 14 time-series features. Second, we deploy 8 dedicated stacked LSTM neural networks built in Keras 3. Each network uses an Input layer of shape (30, 1), an LSTM layer of 64 units with return sequences, 20% dropout, a second LSTM of 32 units, 20% dropout, Dense-16 ReLU, and a linear Dense-1 output. We compile with Adam and Huber loss to handle non-linear attack bursts, using EarlyStopping to restore best weights.

Third, our forecasting service implements recursive multi-step rollout across 7, 14, and 30-day horizons, deriving expanding 95% confidence intervals based on test residual standard deviation and calculating a multi-factor risk score from 0 to 100. Our test evaluations show strong predictive performance—for example, Malware achieved an MAE of 24.25 and R2 of 0.576, and SQL Injection achieved an MAE of 13.66 and R2 of 0.566. For Phishing, we observed a negative R2 due to extreme quarterly blitz variance, highlighting real-world burst dynamics.

Finally, our backend is built on FastAPI with Motor and MongoDB, providing compound-indexed queries, JWT authentication, and automated OpenAPI documentation, connected to a responsive React 19 and Chart.js SOC dashboard. In summary, this project delivers an end-to-end, mathematically grounded platform for proactive cybersecurity intelligence."

33. COMPREHENSIVE VIVA QUESTIONS & MODEL ANSWERS

Below are 55 rigorous technical questions categorized across all aspects of the project, complete with precise model answers verified against the repository codebase:

Q1 [Basic]: What is the primary objective of this project?

Technically Rigorous Answer: To forecast future daily incident volumes across 8 distinct cyber threat categories over 7, 14, and 30-day horizons using stacked LSTM neural networks, transforming reactive SOC operations into proactive risk management.

Q2 [Basic]: What are the 8 threat categories supported by the system?

Technically Rigorous Answer: Phishing, Malware, Brute Force, DDoS, Botnet, SQL Injection, Ransomware, and Data Breach, defined in `backend/app/config.py:SUPPORTED_THREAT_TYPES`.

Q3 [Basic]: What is the difference between reactive and predictive cybersecurity?

Technically Rigorous Answer: Reactive cybersecurity detects attacks after they enter the network or while occurring (t = 0). Predictive cybersecurity uses historical time-series patterns to anticipate volume surges days or weeks in advance (t + k).

Q4 [Basic]: Who are the target end-users of this platform?

Technically Rigorous Answer: Tier-1/Tier-2/Tier-3 SOC Analysts, Threat Intelligence Specialists, CISOs, and Network Security Engineers responsible for enterprise security monitoring.

Q5 [Basic]: Is this system intended to replace existing SIEM/IDS appliances?

Technically Rigorous Answer: No. It complements SIEM/IDS appliances. SIEM tools detect granular packet/log anomalies in real time, while this system provides macro-level capacity planning and proactive threat anticipation.

Q6 [FastAPI]: Why did you choose FastAPI over Flask or Django?

Technically Rigorous Answer: FastAPI provides native asynchronous ASGI performance via Starlette and Uvicorn, automatic data validation via Pydantic v2, and automatic Swagger/OpenAPI documentation generation at `/api/docs` with zero extra code.

Q7 [FastAPI]: How is the application lifespan managed in your FastAPI code?

Technically Rigorous Answer: Using `@asynccontextmanager async def lifespan(app: FastAPI)` in `main.py`. It initiates the Motor MongoDB connection pool and verifies compound indexes on startup, cleanly closing all database connections on shutdown.

Q8 [FastAPI]: How do you handle cross-origin requests between Vite and FastAPI?

Technically Rigorous Answer: Via FastAPI's `CORSMiddleware` in `main.py`, whitelisting `settings.CORS_ORIGINS` (ports 5173 and 3000), while Vite's `vite.config.js` sets up a development reverse proxy forwarding `/api` to port 8000.

Q9 [Python]: What role does Pydantic play in this architecture?

Technically Rigorous Answer: Pydantic parses, validates, and serializes all API request and response data (defined in `schemas.py`). It enforces strict data types, date formatting, and value bounds (e.g., `forecast_days` between 1 and 90).

Q10 [FastAPI]: How does the global exception handler work?

Technically Rigorous Answer: In `main.py`, `@app.exception_handler(Exception)` intercepts unhandled exceptions, logs the full error traceback to console, and returns an HTTP 500 JSON response with a standardized error envelope.

Q11 [MongoDB]: Why did you choose MongoDB instead of a relational database like PostgreSQL?

Technically Rigorous Answer: Cyber incident telemetry is inherently semi-structured and high-volume. MongoDB's document model allows flexible incident schemas with nested metadata, while the Motor async driver integrates natively with FastAPI's event loop.

Q12 [MongoDB]: What collections are defined in your MongoDB database?

Technically Rigorous Answer: Five collections in database `cyber_threat_forecast`: `threat_events` (raw/ingested logs), `forecasts` (cached inference points), `datasets` (file metadata), `models` (metadata/metrics), and `users` (credentials).

Q13 [MongoDB]: What indexes did you create and why are they necessary?

Technically Rigorous Answer: Compound index `(date: 1, threat_type: 1)` on `threat_events` to accelerate daily time-series aggregations, unique index on `email` in `users`, and unique index on `threat_type` in `models`. Without them, group-by queries would trigger slow full-collection scans.

Q14 [MongoDB]: What driver connects Python to MongoDB asynchronously?

Technically Rigorous Answer: Motor (`motor.motor_asyncio.AsyncIOMotorClient`), which wraps PyMongo to provide non-blocking asynchronous I/O that prevents database queries from freezing the FastAPI worker process.

Q15 [MongoDB]: How are forecast points persisted in MongoDB?

Technically Rigorous Answer: In `forecast.py`, upon generating predictions, stale records for that threat and horizon are purged, and newly generated points are inserted via `db.forecasts.insert_many()` for historical review and fast caching.

Q16 [Security]: How are user passwords secured in your database?

Technically Rigorous Answer: Using direct `bcrypt` password hashing with 12 salt rounds (`bcrypt.gensalt(12)`). Plaintext passwords are truncated to 72 bytes and hashed via `bcrypt.hashpw()`; passwords are never stored in plaintext.

Q17 [Security]: How does JSON Web Token (JWT) authentication work in your application?

Technically Rigorous Answer: Upon login, `create_access_token()` creates an RFC 7519 signed JWT token using HMAC-SHA256 (`HS256`) containing user email, role, and a 24-hour expiration. The frontend stores it in `localStorage` and transmits it in the `Authorization: Bearer ` header.

Q18 [Security]: How do you protect endpoints from unauthorized access?

Technically Rigorous Answer: Using FastAPI's dependency injection: `get_current_user` decodes and verifies the Bearer JWT. For elevated actions, `require_role(['admin'])` verifies the user's role before allowing execution.

Q19 [Security]: What roles are implemented in your Role-Based Access Control (RBAC)?

Technically Rigorous Answer: Three roles: `analyst` (standard operational access to forecast and view telemetry), `admin` (elevated privileges to upload datasets and retrain models), and `viewer` (read-only audit access).

Q20 [Security]: Where are cryptographic secrets stored?

Technically Rigorous Answer: In environment configuration (`.env`) loaded via `pydantic-settings` in `config.py`. Production secrets must never be committed to source control.

Q21 [ML]: What is an LSTM and how does it differ from a standard RNN?

Technically Rigorous Answer: Long Short-Term Memory is a specialized recurrent neural network architecture designed by Hochreiter and Schmidhuber. It resolves the vanishing gradient problem of standard RNNs using a cell state regulated by three affine gates: Forget, Input, and Output.

Q22 [ML]: What is the exact layer architecture of your LSTM model?

Technically Rigorous Answer: Input layer `(30, 1)` -> LSTM (64 units, return_sequences=True) -> Dropout (0.20) -> LSTM (32 units, return_sequences=False) -> Dropout (0.20) -> Dense (16 units, ReLU) -> Dense (1 unit, Linear).

Q23 [ML]: What is the total number of trainable parameters in your model?

Technically Rigorous Answer: 29,857 trainable parameters (~116.6 KB in memory), which prevents memorization of noise while enabling sub-15ms inference latency on CPU.

Q24 [ML]: Why did you use Huber loss instead of Mean Squared Error (MSE)?

Technically Rigorous Answer: MSE squares errors, leading to explosive gradient updates during extreme cyber attack burst spikes (such as a 4.5x volumetric DDoS flood). Huber loss behaves quadratically for small errors (|e| <= 1.0) and linearly for large errors, providing robustness against burst outliers.

Q25 [ML]: What is the sequence length (lookback window) and why was 30 chosen?

Technically Rigorous Answer: 30 days (`DEFAULT_SEQUENCE_LENGTH = 30`). This provides exactly one full calendar month of historical context, capturing four full weekly cycles and monthly business rhythms needed to prime the recurrent state.

Q26 [ML]: How did you prevent overfitting during training?

Technically Rigorous Answer: Through two mechanisms: (1) Dropout layers with rate 0.20 after each LSTM layer to prevent co-adaptation of recurrent units, and (2) Keras `EarlyStopping` monitoring validation loss with a patience of 7 epochs to restore the best weights.

Q27 [ML]: Why did you normalize the data using MinMaxScaler?

Technically Rigorous Answer: LSTM internal cell states use `tanh` (range -1 to 1) and `sigmoid` (range 0 to 1) activation functions. Unnormalized counts (e.g., 450 incidents) would saturate the activations, causing vanishing gradients. MinMaxScaler scales all inputs to `[0, 1]`.

Q28 [ML]: How is the data split between training and testing?

Technically Rigorous Answer: Chronologically: the first 80% (560 days) is used for training, and the final 20% (140 days) is held out for testing. Random shuffling is strictly prohibited to avoid lookahead data leakage.

Q29 [ML]: What optimizer and learning rate did you select?

Technically Rigorous Answer: Adam (Adaptive Moment Estimation) optimizer with a learning rate of 0.003, providing fast convergence and adaptive per-parameter learning rate adjustments.

Q30 [ML]: What is the batch size and upper epoch limit?

Technically Rigorous Answer: Batch size is 16 samples; upper epoch limit is 30 epochs, though EarlyStopping typically restores optimal weights between epochs 18 and 26.

Q31 [Forecasting]: How does the recursive multi-step forecasting algorithm work?

Technically Rigorous Answer: To forecast h days ahead, the model takes the initial 30-day window, predicts day t+1, shifts the window forward by dropping day t-29 and appending day t+1, and repeats this process h times. The predictions are then inverse-scaled back to raw incident counts.

Q32 [Forecasting]: What are the supported forecast horizons?

Technically Rigorous Answer: 7 days (tactical sprint), 14 days (operational window), and 30 days (strategic horizon), verified in `backend/app/config.py:DEFAULT_FORECAST_HORIZONS`.

Q33 [Forecasting]: How do you calculate the 95% confidence interval?

Technically Rigorous Answer: Using `Margin = 1.96 * residual_std * sqrt(1 + t/30)`. The `sqrt(1 + t/30)` term causes the uncertainty cone to expand as the horizon deepens, reflecting accumulated autoregressive error. Lower bounds are clamped at 0.0.

Q34 [Forecasting]: What is `residual_std` and where does it come from?

Technically Rigorous Answer: It is the standard deviation of prediction residuals `(y_true - y_pred)` measured on the 140-day test set during model evaluation, stored in `backend/models/lstm/*_metrics.json`.

Q35 [Forecasting]: Can predicted incident counts ever be negative?

Technically Rigorous Answer: No. In `forecast.py`, `pred_counts = np.clip(np.round(pred_counts, 2), 0.0, None)` and `lower_bound = max(0.0, pred - margin)` strictly prevent negative values.

Q36 [Risk]: How does the Composite Risk Score convert predictions into a 0–100 score?

Technically Rigorous Answer: By combining three operational factors: Volume Score (up to 50 pts), Trend Velocity Score (-15 to +25 pts), and Burst Spike Score (up to 25 pts), multiplied by the category severity weight and clipped between 5.0 and 98.0.

Q37 [Risk]: What are the category severity weights and why are they unequal?

Technically Rigorous Answer: Ransomware (1.40), Data Breach (1.35), DDoS (1.25), Malware (1.15), Botnet (1.05), SQL Injection (1.00), Phishing (0.95), and Brute Force (0.85). They reflect organizational impact: a ransomware outbreak is far more destructive than credential scanning.

Q38 [Risk]: What are the four risk categories and their threshold boundaries?

Technically Rigorous Answer: Critical (>= 75.0), High (50.0 to 74.9), Medium (25.0 to 49.9), and Low (< 25.0), verified in `backend/app/ml/forecast.py:calculate_risk_profile()`.

Q39 [Risk]: What constitutes an 'Increasing' trend direction?

Technically Rigorous Answer: A predicted percentage change of +4.0% or greater relative to the 60-day historical baseline. Changes <= -4.0% are 'Decreasing'; otherwise 'Stable'.

Q40 [Risk]: How does the `/api/analytics/overview` endpoint optimize risk score delivery?

Technically Rigorous Answer: It uses an in-memory dictionary cache with a 300-second (5-minute) TTL (`_risk_cache`), preventing expensive redundant neural inference on every dashboard refresh.

Q41 [Dataset]: Is your historical dataset real or synthetic?

Technically Rigorous Answer: Synthetic. It was generated via `backend/scripts/generate_demo_data.py` to simulate realistic cyber attack dynamics (trends, weekly cycles, quarterly blitzes, multi-day bursts, and noise) across 730 continuous days (17,678 records).

Q42 [Dataset]: What happens if there are missing dates in the raw logs?

Technically Rigorous Answer: The preprocessor executes `grouped.reindex(continuous_idx, fill_value=0)`, creating a continuous daily date index and filling missing dates with zero counts so the LSTM sequence integrity is never broken.

Q43 [Dataset]: What are the 14 engineered features in the preprocessor?

Technically Rigorous Answer: Count, day_of_week, day_of_month, month, is_weekend, rolling_mean_7, rolling_std_7, rolling_mean_14, rolling_mean_30, lag_1, lag_7, lag_14, ema_7, and daily_change.

Q44 [Dataset]: Does the deployed LSTM model use all 14 engineered features?

Technically Rigorous Answer: No. The deployed model is strictly a **univariate** model taking an input shape of `(Batch, 30, 1)` (incident count). The 14 features were created for exploratory data analysis and future multivariate extensions.

Q45 [Dataset]: How does the system handle CSV dataset uploads via the web UI?

Technically Rigorous Answer: The file is sent as multipart FormData to `/api/datasets/upload`. `validate_csv_dataset()` checks for required headers (Date, Threat_Type, Severity, Count), validates date formats, ensures non-negative counts, and batch-inserts records into MongoDB.

Q46 [Frontend]: What library is used for charting and why?

Technically Rigorous Answer: Chart.js (via `react-chartjs-2`), utilizing HTML5 canvas rendering for smooth rendering of 90+ point continuous dual-axis lines, filled confidence cones, and responsive tooltips.

Q47 [Frontend]: How does the frontend bridge historical and forecast data on the chart?

Technically Rigorous Answer: In `ForecastView.jsx`, the historical array is rendered with a solid cyan line, and the forecast array is rendered with a dashed purple line. A shared bridge point connects them seamlessly without visual gaps.

Q48 [Frontend]: How does the frontend communicate with the backend?

Technically Rigorous Answer: Through `frontend/src/api.js` using native `fetch()`. It proxies requests to `http://127.0.0.1:8000` via Vite, automatically attaching JWT Bearer tokens from `localStorage`.

Q49 [Frontend]: What state management is used in the React application?

Technically Rigorous Answer: Clean React component state hooks (`useState`, `useEffect`) managed at the `App.jsx` root level, lifting state where needed without the boilerplate of Redux.

Q50 [Frontend]: How does the live heartbeat telemetry dot work?

Technically Rigorous Answer: `App.jsx` runs an automated `setInterval` hook every 30 seconds querying `/api/health`. If the database is connected, the pulse dot turns glowing green (`LIVE TELEMETRY`); otherwise orange (`CONNECTING`).

Q51 [Testing]: What automated tests exist in the project?

Technically Rigorous Answer: A Pytest suite across three phases: `test_phase1.py` (configuration, dataset, MongoDB connection), `test_phase2.py` (preprocessing, sequence shape, LSTM architecture, 8 models existence), and `test_phase3.py` (API endpoints, auth, upload validation).

Q52 [Testing]: How do you verify frontend-backend integration?

Technically Rigorous Answer: Via `backend/tests/verify_frontend_integration.py`, which uses HTTPX to execute automated end-to-end calls through the Vite proxy (port 5173) to FastAPI (port 8000).

Q53 [Difficult]: Why does the Phishing model have a negative R^2 (-0.0312)?

Technically Rigorous Answer: Because the test split coincided with severe, erratic quarterly blitz bursts (reaching 342 attacks/day). The squared residual errors of the smooth LSTM predictions exceeded the total baseline variance, resulting in a negative R^2. It demonstrates that Phishing requires exogenous spam-volume features.

Q54 [Difficult]: Can this system detect zero-day attacks?

Technically Rigorous Answer: No. This system is a macro-level time-series volume forecaster, not an intrusion detection payload inspector. It forecasts expected incident numbers, not individual packet exploit signatures.

Q55 [Difficult]: What is the biggest limitation of this project and how would you fix it?

Technically Rigorous Answer: The univariate nature of the model and recursive error accumulation over 30 days. To fix it, I would transition to a Temporal Fusion Transformer (TFT) that ingests the 14 engineered features alongside live SIEM streaming data via Apache Kafka.

34. COMPREHENSIVE TECHNICAL GLOSSARY

Term / Acronym	Technical Definition & Context in CyberGuard AI
Autoregressive Model	A statistical or neural model that predicts future values based on past values of the same variable via a recursive sliding window.
bcrypt	An adaptive cryptographic password-hashing function based on the Blowfish cipher with configurable work factor (12 salt rounds in this project).
Confidence Interval	A statistical range around a forecast within which the true value is expected to fall with a specified probability (95% in this system).
CORS	Cross-Origin Resource Sharing; security mechanism allowing the React frontend (port 5173) to access the FastAPI backend (port 8000).
Cosine / Sine Seasonality	Mathematical harmonics used in `generate_demo_data.py` to model cyclical patterns like quarterly campaigns.
Data Leakage	Accidentally sharing information between training and test sets. Prevented here by strict chronological 80/20 train/test splitting.
DDoS	Distributed Denial of Service; volumetric cyber attack attempting to overwhelm network capacity, modeled here with 1.25x severity weight.
Dropout	Regularization technique deactivating random neurons (20% in this system) during training passes to prevent overfitting.
EarlyStopping	Callback terminating training when validation loss stops improving for a specified patience (7 epochs), restoring optimal weights.
EMA	Exponential Moving Average; a moving average giving exponential weight to recent daily observations to track short-term spikes.
FastAPI	High-performance asynchronous Python web framework built on Starlette and Pydantic, providing automatic Swagger OpenAPI documentation.
Huber Loss	Loss function combining the quadratic advantages of MSE for small errors with the robust linear penalty of MAE for burst outliers.
JSON Web Token (JWT)	An open RFC 7519 standard defining a compact, self-contained method for securely transmitting claims between parties as a signed JSON object.
Keras 3	High-level deep learning API running on top of TensorFlow, used here to build, compile, train, and serialize stacked LSTM networks.
Lag Feature	A historical value from a fixed prior time step (e.g., lag_1 = yesterday, lag_7 = same day last week) used in autoregression.
Lifespan Context	FastAPI's modern async context manager managing resource startup (MongoDB connection pool) and shutdown cleanup.
LSTM	Long Short-Term Memory; a specialized recurrent neural network capable of learning long-term sequential dependencies via gating mechanisms.
MAE	Mean Absolute Error; average magnitude of forecast errors in incident count units without penalizing outliers quadratically.
MAPE	Mean Absolute Percentage Error; average percentage discrepancy between predicted and actual attack counts.
MinMaxScaler	Feature scaling algorithm transforming raw counts into the range [0, 1] to prevent recurrent neuron saturation.
MongoDB	NoSQL document database storing unstructured threat logs, cached forecasts, model registry documents, and user accounts.
Motor	Official asynchronous, non-blocking Python driver for MongoDB integrating directly into FastAPI's asyncio event loop.
OAuth2 Bearer Token	Authentication standard where the client proves authorization by providing an issued JWT in the HTTP Authorization header.
Pydantic v2	Data validation and settings library using Python type hints, rewritten in Rust for high-speed serialization and parsing.
R^2 Score	Coefficient of determination; measures the proportion of variance in target attack volume explained by the LSTM model.
RBAC	Role-Based Access Control; authorizing permissions based on assigned user roles (analyst, admin, viewer).
Residual Standard Deviation	Standard deviation of prediction errors on the test set, used as the baseline multiplier for dynamic confidence intervals.
RMSE	Root Mean Squared Error; square root of average squared errors, heavily penalizing large attack surge prediction misses.

SIEM	Security Information and Event Management; enterprise software aggregating logs from firewalls, servers, and network devices.
SOC	Security Operations Center; centralized facility where cybersecurity staff monitor, detect, analyze, and investigate security incidents.
Time-Series Continuity	Ensuring no missing dates exist in the timeline; enforced via `grouped.reindex()` filling missing days with zero counts.
Uvicorn	Lightning-fast ASGI web server implementation for Python, powering the concurrent execution of the FastAPI application.
Vite	Modern frontend build tool offering instant development server start, rapid Hot Module Replacement, and optimized Rollup bundling.

35. FINAL PROJECT SUMMARY & CONCLUSIONS

What This Project Actually Accomplishes:
The Cyber Threat Forecasting System represents an operational, fully audited academic final year project. It successfully proves that recurrent neural architectures (specifically 2-layer stacked LSTMs trained with Huber loss) can capture complex non-linear patterns within cyber attack telemetry. By deploying 8 dedicated models across 8 distinct threat vectors, the platform respects the distinct behavioral characteristics of diverse cyber attacks—from work-hour phishing blitzes to weekend volumetric DDoS floods.

Architectural Takeaways:
• **Backend:** FastAPI and Motor provide high-concurrency, asynchronous non-blocking performance with automated Swagger self-documentation.
• **Database:** MongoDB maintains 5 compound-indexed collections for high-speed time-series filtering and instant forecast caching.
• **Security:** Cryptographically sound bcrypt (12 rounds) password storage and signed HS256 JWT tokens enforce role-based access.
• **Frontend:** React 19 and Chart.js deliver an intuitive, glassmorphism-styled SOC workstation with interactive dual-axis charts and risk meters.
• **Machine Learning:** Compact 29,857-parameter LSTM networks deliver sub-15ms predictions, 95% expanding confidence cones, and a 0–100 composite risk matrix.

Armed with this complete technical documentation, the project candidate possesses the verified facts, architectural diagrams, exact metrics, and critical code explanations needed to defend this project confidently before university evaluators, external examiners, and enterprise technical interviewers.